ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC QUỐC TẾ

BÁO CÁO ĐỒ ÁN

MÔN: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Dự án: *Plants vs Zombies: Golden Spatula* (Greenfoot Framework)

Tác giả: Lê Anh Kiệt

Giảng viên: L.D.Tân, N.Q.Phú

Ngày 27 tháng 4 năm 2026

Tóm tắt nội dung

Bản báo cáo này phân tích kiến trúc phần mềm và hệ thống thực thể trong dự án *PvZ: Golden Spatula*. Bằng cách vận dụng các nguyên lý lập trình hướng đối tượng (OOP) trên nền tảng Greenfoot. Tôi đã xây dựng một hệ thống vận hành linh hoạt, tối ưu hóa việc quản lý thực thể đa dạng và các cơ chế tương tác phức tạp. Nội dung tiếp theo sẽ đi sâu vào cấu trúc hạ tầng điều phối và logic hành vi của hệ thống.

Thành viên nhóm

STT	Họ và Tên	MSSV	Vai trò	Trách nhiệm chính
1	Lê Anh Kiệt	ITCSIU24047	All	
2	Dương Gia Thục	ITITWE23026	Zombie	Xây dựng các lớp con Zombie, stats, hoạt ảnh và spawn wave.
3	Nguyễn Quốc Trung Kiên	ITCSIU24034	Zombie	Xây dựng các lớp con Plant, cơ chế đạn và đặt cây trên lưới.
4	Lê Thanh Hoàng	ITCSIU24034	Zombie	Xây dựng các lớp con Plant, chi phí/hồi chiêu và hoạt ảnh.
5	Nguyễn Viết Huy	ITCSIU24034	Zombie	Xây dựng các lớp con Plant, sản xuất mặt trời và xử lý va chạm.

Mục lục

1 Giới thiệu **8**

2 Tổng quan về trò chơi **8**

3 Kiến trúc hệ thống **8**
3.1 Cấu trúc Package 8
3.2 Trách nhiệm của các lớp 9

4 Nguyên tắc SOLID **10**

5 Design Patterns (Mẫu thiết kế) **11**

6 GridManager **12**
6.1 Giới Thiệu 12
6.2 Cấu Trúc Dữ Liệu 12
6.3 Thuật Toán Hình Học Lưới Lục Giác 13
6.4 Logic Kiểm Tra Đặt Cây (`canPlace`) 13
6.5 Kết Luận Sơ Bộ 14

7 Arena: Thiết Kế Màn Hình Khởi Đầu và Hiệu Ứng Dẫn Truyện **15**
7.1 Vai Trò và Chức Năng 15
7.2 Cấu Trúc Dữ Liệu Đợt Tấn Công (Wave Matrix) 15

8 SeedBank: Hệ Thống Quản Lý Thẻ Bài và Cơ Chế Kéo-Thả **17**
8.1 Vai Trò và Chức Năng Chính 17
8.2 Thuật Toán Xử Lý Kéo-Thả (Drag-and-Drop Logic) 17
8.3 Quản Lý Tài Nguyên (Economy System) 17
8.4 Cập Nhật Cửa Hàng Động (Bank Refreshment) 17
8.5 Phản Hồi Trực Quan (Visual Feedback) 18

9 UpgradeManager: Quản Lý Logic Tiến Hóa Thực Thể **19**
9.1 Kiến Trúc Lớp Tiện Ích (Utility Class Design) 19
9.2 Cơ Chế Phân Loại Thực Thể Bằng Polymorphism 19
9.3 Thuật Toán Quyết Định Kết Quả (Factory Method Pattern) 19
9.4 Khả Năng Mở Rộng và Bảo Trì 19
9.5 Đánh Giá Kỹ Thuật 20

10 WaveManager: Hệ Thống Điều Phối Đợt Tấn Công và Sự Kiện **21**
10.1 Vai Trò và Tầm Quan Trọng 21
10.2 Thuật Toán Quản Lý Thời Gian Thực (Delta Time) 21
10.3 Cơ Chế Phân Loại Đợt Tấn Công 21
10.4 Thuật Toán Sinh Quái và Định Vị Làn (Lane Positioning) 21
10.5 Hệ Thống Phản Hồi và Thông Báo 21
10.6 Điều Kiện Chiến Thắng (Win Condition) 22

11 PlayScene **23**
11.1 Thiết Kế Kiến Trúc Hệ Thống . . . 23
11.2 Phân Tích Cơ Chế Core-Logic . . . 23
11.3 Tối Ưu Hóa Hiệu Năng . . . 24
11.4 Kết Luận . . . 24

12 RollButton **25**
12.1 Giới Thiệu . . . 25
12.2 Thiết kế Giao diện (Rendering) . . . 25
12.3 Thuật toán Xác suất Trọng số (Weighted Random) . . . 26
12.4 Luồng xử lý sự kiện (Event Flow) . . . 26
12.5 Kết Luận . . . 27

13 RupButton **28**
13.1 Cấu Trúc Dữ Liệu và Thành Phần . . . 28
13.2 Logic Nâng Cấp và Tiến Hóa Xác Suất . . . 28
13.3 Giao Diện và Phản Hồi Người Chơi . . . 29
13.4 Kết Luận . . . 29

14 AugmentCard **30**
14.1 Vai Trò trong Hệ Thống Gameplay . . . 30
14.2 Thuật Toán Xử Lý Tương Tác Chuột (UX Animation) . . . 30
14.3 Cơ Chế Áp Dụng Hiệu Ứng (Augment Logic) . . . 30
14.4 Quản Lý Luồng Sự Kiện (Event Flow) . . . 30

15 PhysicsBody **32**
15.1 Giới Thiệu . . . 32
15.2 Cấu Trúc Dữ Liệu và Định Nghĩa . . . 32
15.3 Thuật Toán Di Chuyển Lượng Giác . . . 33
15.4 Ứng Dụng Trong Logic Game . . . 33
15.5 Kết Luận Sơ Bộ . . . 33

16 SpriteAnimator **35**
16.1 Cơ Chế Quản Lý Thời Gian (Delta Time) . . . 35
16.2 Xử Lý Tài Nguyên Hình Ảnh . . . 35
16.3 Ưu Điểm Kỹ Thuật . . . 36
16.4 Kết Luận Sơ Bộ . . . 36

17 Plant **37**
17.1 Cơ Chế Hợp Nhất (Merging System) . . . 37
17.2 Xử Lý Tương Tác Người Dùng . . . 37
17.3 Vòng Đời Thực Thể (Lifecycle) . . . 38
17.4 Kết Luận Sơ Bộ . . . 38

18 Peashooter **39**
18.1 Cơ Chế Cường Hóa (Plant Food System) . . . 39
18.2 Tối Ưu Hóa Hiệu Năng . . . 39
18.3 Logic Kiểm Tra Mục Tiêu Hiện Đại . . . 40
18.4 Kết Luận Sơ Bộ . . . 40

19 Repeater **41**
19.1 Logic Điều Khiển Nhịp Độ Tấn Công . . . 41
19.2 Quản Lý Hoạt Ảnh và Trạng Thái . . . 41
19.3 Kết Luận Sơ Bộ . . . 42

20 GatlingPea **43**
20.1 Cơ Chế Bắn Chuỗi Tốc Độ Cao . . . 43
20.2 Xử Lý Phản Hồi Tương Tác . . . 43
20.3 Kết Luận Sơ Bộ . . . 44

21 GatlingPea2 **45**
21.1 Cơ Chế Tấn Công Theo Đợt (Burst Logic) . . . 45
21.2 Cơ Chế Cường Hóa Plant Food . . . 45
21.3 Phân Tích Hiệu Năng và Đồ Họa . . . 46
21.4 Kết Luận Sơ Bộ . . . 46

22 Cactus **47**
22.1 Logic Điều Khiển Nhịp Độ Chiến Đấu . . . 47
22.2 Xử Lý Hoạt Ảnh và Phản Hồi . . . 47
22.3 Thuật Toán Nhận Diện Kẻ Thù . . . 48
22.4 Kết Luận Sơ Bộ . . . 48

23 Cactus2 **49**
23.1 Cơ Chế Bắn Đa Hướng (Multi-directional Logic) . . . 49
23.2 Hệ Thống Cường Hóa và Quản Lý Trạng Thái . . . 49
23.3 Tối Ưu Hóa Hệ Thống . . . 50
23.4 Kết Luận Sơ Bộ . . . 50

24 Cactus3 **51**
24.1 Cơ Chế Đạn Đạo Hình Quạt (Spread-shot Mechanism) . . . 51
24.2 Thuật toán Phân rã 3 tia . . . 51
24.3 Cường Hóa Trạng Thái Tối Thượng . . . 51
24.4 Tối Ưu Hóa Truy Vấn và Hiệu Năng . . . 52
24.5 Kết Luận Sơ Bộ . . . 52

25 BonkChoy1 **53**
25.1 Cấu Trúc Hoạt Ảnh và Tiền Xử Lý . . . 53
25.2 Hệ thống Chiến đấu liên hoàn (Combo System) . . . 53
25.3 Cơ chế Hồi phục và Duy trì . . . 53
25.4 Kết Luận Sơ Bộ . . . 54

26 BonkChoy2 **55**
26.1 Sự Khác Biệt về Hiệu Năng Chiến Đấu . . . 55
26.2 Logic Combo và Hồi Phục Nâng Cao . . . 55
26.3 Xử Lý Hoạt Ảnh Đa Trạng Thái . . . 55
26.4 Kết Luận Sơ Bộ . . . 56

27 PotatoMine **57**
27.1 Cơ Chế Kích Ngòi (Arming Logic) . . . 57
27.2 Logic Va Chạm và Kích Nổ . . . 57
27.3 Tương Tác Đặc Biệt . . . 58

27.4 Kết Luận Sơ Bộ . . . 58

28 Zombie **59**
28.1 Máy trạng thái và Logic hành vi . . . 59
28.2 Cơ chế Tương tác Hệ thống . . . 59
28.3 Xử lý Tử vong đa giai đoạn . . . 60
28.4 Kết luận kỹ thuật . . . 60

29 BasicZombie **61**
29.1 Cơ chế Biến đổi Hình thái (Morphology Logic) . . . 61
29.2 Đa hình và Ghi đè phương thức (Method Overriding) . . . 61
29.3 Thông số Kỹ thuật và Cân bằng . . . 62
29.4 Kết luận kỹ thuật . . . 62

30 Brickhead **63**
30.1 Cơ chế Giáp đa tầng và Phân rã vật lý . . . 63
30.2 Thiết kế Code và Tối ưu hóa Hành vi . . . 63
30.3 Thông số Kỹ thuật và Cân bằng Game . . . 64
30.4 Kết luận kỹ thuật . . . 64

31 Buckethead **65**
31.1 Cơ chế Giáp và Phân rã vật lý . . . 65
31.2 Hệ thống ngưỡng hư hại (Degradation Thresholds) . . . 65
31.3 Thiết kế Logic điều khiển . . . 65
31.4 Thông số Kỹ thuật . . . 65
31.5 Kết luận . . . 65

32 Conehead **66**
32.1 Cơ chế Giáp và Phân rã vật lý . . . 66
32.2 Thiết kế Logic điều khiển . . . 67
32.3 Thông số Kỹ thuật . . . 67
32.4 Kết luận . . . 67

33 Projectile **68**
33.1 Cơ Chế Va Chạm và Tối Ưu Hiệu Năng . . . 68
33.2 Xử Lý Hoạt Ảnh Va Chạm . . . 68
33.3 Quản Lý Tài Nguyên . . . 69
33.4 Kết Luận Sơ Bộ . . . 69

34 ProjectileVariants **70**
34.1 Tóm Tắt . . . 70
34.2 Phân loại và Chỉ số Kỹ thuật . . . 70
34.3 Cơ chế Đạn xuyên thấu và Quỹ đạo phức tạp . . . 70
34.4 So sánh Hiệu quả Chiến thuật . . . 71
34.5 Kết Luận Sơ Bộ . . . 71

35 Explosion **72**
35.1 Cơ chế Sát thương và Tương tác . . . 72
35.2 Quản lý Hoạt ảnh và Vòng đời . . . 72
35.3 Thông số Kỹ thuật Tóm lược . . . 72
35.4 Kết Luận Sơ Bộ . . . 73

36 FallingObject **74**
36.1 Mô Hình Vật Lý và Quỹ Đạo 74
36.2 Quản Lý Vòng Đời và Hiệu Ứng 74
36.3 Ứng Dụng Thực Tế 75
36.4 Kết Luận Sơ Bộ 75

37 PhysicalEntities **76**
37.1 Tóm lược nội dung 76
37.2 Phân tích Tham số Động lực học 76
37.3 Cơ chế Mô phỏng Quỹ đạo Văng 76
37.4 Triển khai Lập trình Hướng đối tượng 76
37.5 Kết luận kỹ thuật 77

38 SunMechanics **78**
38.1 Tóm Tắt 78
38.2 Logic Thu thập và Di chuyển 78
38.3 Quản lý Vòng đời và Trạng thái 78
38.4 Sự khác biệt trong Vật lý học 79
38.5 Kết Luận Sơ Bộ 79

39 fallingZombie **80**
39.1 Tóm Tắt 80
39.2 Cơ chế Hoạt ảnh và Hiệu ứng Mờ dần 80
39.3 Quản lý Vòng đời Đối tượng 80
39.4 Thông số Kỹ thuật 80
39.5 Kết Luận Sơ Bộ 81

40 Tổng kết và Đánh giá **82**

1 Giới thiệu

Plants vs Zombies là một trò chơi thuộc thể loại thủ thành (tower-defense), nơi người chơi sử dụng các loại cây trồng để bảo vệ khu vườn khỏi các đợt tấn công của thây ma (zombies). Mục tiêu chính của dự án này là tạo ra một bản mô phỏng chức năng của trò chơi Plants vs Zombies (PVZ) bằng cách sử dụng **Greenfoot** và các nguyên tắc **Lập trình hướng đối tượng (OOP)**.

1. **Công nghệ sử dụng:** Trò chơi được xây dựng hoàn toàn bằng ngôn ngữ Java trên nền tảng Greenfoot. Greenfoot được lựa chọn nhờ mô hình actor-world trực quan, hệ thống quản lý vòng lặp game tích hợp sẵn và hỗ trợ mạnh mẽ cho cấu trúc phân cấp lớp, là môi trường lý tưởng để thể hiện thiết kế OOP.

2. **Phạm vi dự án:** Dự án đã triển khai thành công các yếu tố cốt lõi bao gồm: `GameWorld` chính, màn hình `GameOverWorld`, các lớp Actor cho thực vật và thây ma, cơ chế đạn bắn (`PeaProjectile`/`FirePeaProjectile`), hệ thống thu thập mặt trời và chi phí, giao diện HUD, các đợt tấn công của zombie theo làn, và hệ thống phòng thủ máy cắt cỏ. Các tính năng phát hiện va chạm, cập nhật trạng thái game và vòng lặp 'act' đã được tích hợp đầy đủ.

2 Tổng quan về trò chơi

Trò chơi diễn ra trên một lưới cố định. Người chơi cần đặt cây một cách chiến thuật để chống lại các đợt sóng zombie. Các cơ chế cốt lõi xoay quanh việc quản lý tài nguyên và vị trí đặt quân.

- **Đặt cây trên lưới:** Cây được đặt vào các ô lưới, mỗi cây chiếm một ô. Việc đặt cây tiêu tốn "mặt trời"(đơn vị tiền tệ trong game).
- **Hệ thống mặt trời:** Các biểu tượng mặt trời rơi ngẫu nhiên trên màn hình và người chơi phải nhấp chuột để thu thập. Một số loại cây (ví dụ: `Sunflower`) sẽ tạo ra mặt trời định kỳ thông qua phương thức `act()`.
- **Đợt tấn công và Làn đường:** Zombie xuất hiện theo các đợt có thời gian và di chuyển dọc theo các làn đường ngang cố định hướng về phía nhà của người chơi.
- **Đạn và Va chạm:** Cây tấn công bằng cách thêm các Actor đạn vào thế giới. Mỗi viên đạn di chuyển trên làn đường của nó và gọi phương thức `takeDamage()` khi va chạm với bất kỳ zombie nào.
- **Điều kiện kết thúc:** Trò chơi kết thúc khi một zombie vượt qua cột cuối cùng của lưới và máy cắt cỏ ở làn đó đã được sử dụng hết.

3 Kiến trúc hệ thống

3.1 Cấu trúc Package

Dự án sử dụng cấu trúc phân lớp. Tất cả các đối tượng trong game đều kế thừa lớp `Actor` hoặc `World` của Greenfoot. Các hành vi được thực hiện bằng cách **ghi đè (override)** phương thức `act()` kết hợp với các lớp hỗ trợ chuyên biệt.

Package / Lớp	Các lớp chính	Tr
`screens`	`GameWorld`, `GameOverWorld`	Kế và đo
`logic`	`GameState`, `ZombieWaveController`, `HudController`, `LawnMowerController`	Đi tha yếu
`entities`	`Plant`, `Zombie`, `Sun`, `Projectile`, `PeaProjectile`, `FrozenPeaProjectile`	Cá diễ chu
`items`	`SeedBank`, `PlantCard`, `Shovel`	Cá để
`managers`	`BackgroundManager`, `GridConfig`, `FontManager`	Cá tải cụ

Các Interface chính:

- `IGameSpawner` – được sử dụng bởi các bộ điều khiển (ví dụ: `ZombieWaveController`) để trừu tượng hóa việc tạo và đặt các actor mới vào thế giới.
- `ISunReceiver` – được triển khai bởi các lớp xử lý tài nguyên mặt trời thu thập được (ví dụ: `HudController`).

3.2 Trách nhiệm của các lớp

3.2.1 Lớp Plant (Cây)

a. Các loại cây Cây đóng vai trò là các actor phòng thủ cố định. Mỗi loại cây khác nhau về hành vi tấn công, thời gian hồi chiêu và máu (HP).

- **PeaShooter** – tấn công tầm xa cơ bản. (Giá: 100 | Hồi: 2.5s | HP: 100)
- **Repeater** – tạo ra tài nguyên mặt trời. (Giá: 50 | Hồi: 2.5s | HP: 100)
- **Potato Mine** – bẫy nổ có thời gian chờ. (Giá: 25 | Hồi: 10s)
- **GatlingPea** – bắn hai viên đạn mỗi lần tấn công. (Giá: 200 | Hồi: 2.5s | HP: 100)
- **GatlingPea2stars** – bắn hai viên đạn mỗi lần tấn công. (Giá: 200 | Hồi: 2.5s | HP: 100
- **BonkChoy** – bắn hai viên đạn mỗi lần tấn công. (Giá: 200 | Hồi: 2.5s | HP: 100
- **BonkChoy2** – bắn hai viên đạn mỗi lần tấn công. (Giá: 200 | Hồi: 2.5s | HP: 100
- **Cactus** – bắn hai viên đạn mỗi lần tấn công. (Giá: 200 | Hồi: 2.5s | HP: 100
- **Cactus2** – bắn hai viên đạn mỗi lần tấn công. (Giá: 200 | Hồi: 2.5s | HP: 100
- **Cactus3** – bắn hai viên đạn mỗi lần tấn công. (Giá: 200 | Hồi: 2.5s | HP: 100

b. Cấu trúc lớp Plant cơ sở Tất cả các loại cây đều kế thừa từ lớp trừu tượng `Plant`, lớp này kế thừa `Actor` của Greenfoot. Cấu trúc này cho phép xử lý đồng nhất tất cả các loại cây trong các vòng lặp thông qua `World.getObjects(Plant.class)`.

c. Cơ chế tấn công Các cây tấn công duy trì một bộ đếm thời gian hồi chiêu nội bộ giảm dần sau mỗi tick của phương thức `act()`. Khi bộ đếm về 0, cây sẽ thêm một Actor `Projectile` mới vào thế giới. Sát thương được áp dụng khi đạn phát hiện va chạm với zombie.

d. Sản sinh tài nguyên Mặt trời rơi từ trên cao xuống , và chiến thắng mỗi đợt Zombies tạo ra các token mặt trời.

e. Xử lý máu và sát thương Phương thức `takeDamage(int damage)` kiểm tra trạng thái sống/chết của cây trước khi trừ máu. Nếu máu xuống bằng hoặc dưới 0, trạng thái `startDeath` sẽ được kích hoạt.

f. Xẻng và Hoàn tiền Lớp `Shovel` cho phép người chơi gỡ bỏ cây. Khi sử dụng, hệ thống sẽ tính toán tọa độ lưới, xác định loại cây và thực hiện hoàn trả một phần chi phí (theo `SHOVEL_REFUND_RATIO`) thông qua `ISunReceiver`.

3.2.2 Lớp Zombie (Thây ma)

a. Các loại Zombie

- **Normal Zombie:** HP 100, tốc độ 15.
- **Conehead Zombie:** HP 100 + giáp nón 200, tốc độ 18.
- **Buckethead Zombie:** HP 100 + giáp xô 300, tốc độ 18.
- **Brickhead Zombie:** HP 160, tốc độ 32.

b. Cơ chế máu và Giáp Đối với các zombie có giáp, một đối tượng `DamageStrategy` sẽ chặn sát thương trước khi nó ảnh hưởng đến lượng máu cơ bản. Điều này đảm bảo tính đóng gói và dễ dàng thay đổi logic cho các loại zombie đặc biệt khác.

c. Hành vi di chuyển Mỗi zombie di chuyển sang trái trong mỗi lần gọi `act()` với khoảng cách tỉ lệ thuận với tốc độ của nó, trừ khi nó đang trong trạng thái ăn cây (`isEating`) hoặc đang chết (`isDying`).

4 Nguyên tắc SOLID

Thiết kế của dự án tuân thủ cả 5 nguyên tắc SOLID:

- **SRP (Đơn trách nhiệm):** Mỗi lớp chỉ có một lý do để thay đổi (ví dụ: `MovementSystem` chỉ cập nhật vị trí).
- **OCP (Đóng/Mở):** Thêm cây mới chỉ cần tạo lớp con mới mà không cần sửa đổi mã nguồn hiện có.

- **LSP (Thay thế Liskov):** Các lớp con của Plant/Zombie có thể thay thế hoàn toàn cho lớp cha trong các vòng lặp xử lý.
- **ISP (Phân tách Interface):** Các Interface nhỏ gọn như `ISunReceiver`, tránh ép buộc các lớp triển khai các phương thức không cần thiết.
- **DIP (Đảo ngược phụ thuộc):** Các lớp cấp cao phụ thuộc vào các trừu tượng (`IGameSpawner`) thay vì các lớp cụ thể.

5 Design Patterns (Mẫu thiết kế)

1. **Strategy Pattern:** Dùng cho tính toán sát thương của các loại zombie khác nhau (Giáp vs Thường).
2. **Factory Pattern:** Tập trung hóa việc khởi tạo các Actor cây thông qua `PlantFactory`.
3. **State Pattern:** Quản lý các trạng thái nội bộ của Actor (đang đứng, đang tấn công, đang chết) và các hoạt ảnh tương ứng.

6 GridManager

`GridManager` là lớp cốt lõi quản lý toàn bộ bàn chơi trong một tựa game chiến thuật phong cách *Auto Chess*, xây dựng trên framework **Greenfoot (Java)**. Lớp này đảm nhận bốn nhiệm vụ chính:

1. Chuyển đổi tọa độ lưới sang tọa độ pixel màn hình.
2. Xử lý tương tác kéo-thả (*drag and drop*) của người chơi.
3. Kiểm tra logic đặt đơn vị và giới hạn sân chơi.
4. Vẽ phản hồi trực quan dạng lục giác (highlight).

Báo cáo phân tích từng thành phần theo hướng học thuật, bao gồm thuật toán hình học, cấu trúc dữ liệu, và thiết kế hướng đối tượng.

6.1 Giới Thiệu

6.1.1 Bối Cảnh

Greenfoot là môi trường lập trình Java trực quan, trong đó mọi đối tượng trò chơi đều kế thừa từ lớp `Actor`. Trong dự án này, `GridManager` cũng là một `Actor` – không phải để hiển thị nhân vật, mà để **vẽ lớp overlay** (hiệu ứng highlight lục giác) lên toàn màn hình.

Lưới lục giác (*hexagonal grid*) được ưa chuộng trong các game chiến thuật vì mỗi ô có **6 ô lân cận cách đều nhau**, không giống lưới ô vuông (4 hoặc 8 ô lân cận không đồng đều).

6.1.2 Phụ Thuộc

Bảng 1: Các lớp phụ thuộc của `GridManager`

Lớp	Vai trò
`Plant`	Đơn vị cây được đặt lên bàn
`SeedPacket`	Gói hạt giống kéo từ tay người chơi
`UpgradeManager`	Kiểm tra điều kiện nâng cấp
`PlayScene`	World chính, kích hoạt cơ chế gộp 3 cây

6.2 Cấu Trúc Dữ Liệu

6.2.1 Mảng Bàn Chơi

```
public Plant[][] Board = new Plant[6][9];
```

Đây là mảng 2 chiều `[hàng][cột]` kích thước $6 \times 9 = 54$ ô. Mỗi phần tử lưu tham chiếu đến đối tượng `Plant` (hoặc `null` nếu trống).

Thiết kế đặc biệt: Hàng thứ 6 (index 5) được dành riêng làm *Bench* (hàng chờ), không tính vào giới hạn số cây trên sân. Điều này được thể hiện rõ trong `getCurrentPlantCount()`:

```
public int getCurrentPlantCount() {
    int count = 0;
    for (int r = 0; r < 5; r++)   // Bo qua hang index 5 (Bench)
```

```
        for (int c = 0; c < 9; c++)
            if (Board[r][c] != null) count++;
    return count;
}
```

Cơ chế này tách biệt *khu chiến đấu* và *khu dự bị*, tương tự cơ chế Bench trong *Teamfight Tactics* (TFT).

6.3 Thuật Toán Hình Học Lưới Lục Giác

6.3.1 Loại Lưới: Flat-Top Offset

Lưới sử dụng kiểu **flat-top** (cạnh nằm ngang ở trên/dưới) với **odd-row offset** – các hàng lẻ được dịch sang phải nửa ô, tạo ra hình dạng tổ ong quen thuộc.

6.3.2 Công Thức Chuyển Đổi Tọa Độ

```
private final double HEX_W  = HEX_R * Math.sqrt(3);
private final double HEX_H  = HEX_R * 2.0;
private final double STEP_X = HEX_W;
private final double STEP_Y = HEX_H * 0.75;
```

Công thức chuyển đổi tọa độ:

$$X(col, row) = X_0 + col \cdot W_{hex} + \begin{cases} \dfrac{W_{hex}}{2} & \text{nếu } row \text{ lẻ} \\ 0 & \text{nếu } row \text{ chẵn} \end{cases} \tag{1}$$

$$Y(row) = Y_0 + row \cdot \frac{3}{4} H_{hex} \tag{2}$$

6.3.3 Thuật Toán Tìm Ô Gần Nhất (Hit-Testing)

Khi người chơi di chuyển chuột, `getGridX()` và `getGridY()` xác định ô đang được trỏ tới bằng **khoảng cách Euclidean**:

```
double d = Math.hypot(mx - getXCoord(c, r), my - getYCoord(c, r));
if (d < minD) { minD = d; best = c; }
```

6.3.4 Vẽ Lục Giác Highlight

Tọa độ đỉnh thứ i của lục giác:

$$(x_i,\, y_i) = (c_x + R\cos(60^\circ \cdot i - 90),\; c_y + R\sin(60^\circ \cdot i - 90)), \quad i = 0, 1, \ldots, 5 \tag{3}$$

6.4 Logic Kiểm Tra Đặt Cây (canPlace)

```
canPlace(x, y, plant):
  1. Kiem tra bien (out-of-bounds) --> false neu ngoai luoi
  2. Neu dat o 5 hang dau (chien dau):
     a. Neu o trong VA cay chua o tren san --> kiem tra capacity
  3. Cho phep neu:
```

```
    - O trong (target == null)
    - O chua chinh cay nay (di chuyen)
    - O chua cay nang cap duoc (UpgradeManager)
```

6.5 Kết Luận Sơ Bộ

`GridManager` là một cài đặt thực tế và hiệu quả cho bài toán lưới lục giác trong game 2D. Các cải tiến chủ yếu tập trung vào *encapsulation* và tối ưu nhỏ trong hit-testing – những điểm quan trọng cho khả năng bảo trì về sau.

7 Arena: Thiết Kế Màn Hình Khởi Đầu và Hiệu Ứng Dẫn Truyện

7.1 Vai Trò và Chức Năng

Lớp `Arena` đóng vai trò là môi trường trung gian (Pre-game World), chịu trách nhiệm thiết lập bầu không khí cho trò chơi trước khi bắt đầu màn chơi chính. Các nhiệm vụ trọng tâm bao gồm:

1. Khởi tạo cấu trúc đợt tấn công (`WaveManager`) với dữ liệu mảng 3 chiều.
2. Xử lý đoạn hội thoại của nhân vật dẫn chuyện (Crazy Dave) bằng hệ thống ảnh GIF.
3. Thực hiện hiệu ứng camera lia (*Background Scrolling*) để giới thiệu lực lượng quân địch.

7.2 Cấu Trúc Dữ Liệu Đợt Tấn Công (Wave Matrix)

Hệ thống quân địch được định nghĩa thông qua một mảng đa chiều phức tạp, cho phép kiểm soát chi tiết từng đơn vị Zombie xuất hiện theo từng đợt (*Wave*) và từng làn (*Lane*):

Listing 1: Cấu trúc mảng 3 chiều định nghĩa đợt tấn công

```java
public Zombie[][][] level1 = {
    {{new BasicZombie()}, null, null, null, null},
    // ...
    {{new Brickhead()}, {new Buckethead()}, {new Buckethead()}, {
        new Buckethead()}, {new Brickhead()}, null}
};
```

Ma trận này có cấu trúc Waves × Lanes × Zombies per Lane. Việc sử dụng đối tượng trực tiếp bên trong mảng giúp `WaveManager` có thể lấy mẫu thực thể (*Instantiate*) một cách chính xác tại thời điểm thực thi.

7.2.1 Thuật Toán Hiệu Ứng Lia Máy (Smooth Scrolling)

Để tạo cảm giác không gian rộng lớn, `Arena` thực hiện di chuyển đồng bộ giữa hình nền và các đối tượng `IdleZombie`. Thuật toán di chuyển được tính toán dựa trên biến đếm `count`:

$$\Delta X = \begin{cases} -V_{scroll} & \text{nếu } 100 < \text{count} < 160 \text{ (Lia sang phải)} \\ +V_{scroll} & \text{nếu } 350 < \text{count} < 410 \text{ (Quay về)} \\ 0 & \text{các trường hợp còn lại} \end{cases} \tag{4}$$

Phương thức `scrollBGimage(int offset)` cập nhật vị trí của toàn bộ `Actor` hiện diện trên màn hình để đảm bảo tính nhất quán về vị trí tương đối so với bản đồ nền.

7.2.2 Xử Lý Hội Thoại và Hoạt Ảnh GIF

Lớp sử dụng thư viện `GifImage` để giải mã hoạt ảnh của Crazy Dave. Một cơ chế kiểm soát logic được thiết lập để tạm dừng toàn bộ tiến trình game cho đến khi người chơi nhấp chuột (`Greenfoot.mouseClicked`), tạo sự tương tác trực tiếp:

```java
private void handleDave() {
    GreenfootImage currentDave = daveGif.getCurrentImage();
    // Quy doi ty le anh cho phu hop voi do phan giai man hinh
    if (currentDave.getWidth() != 600) {
        currentDave.scale(600, 400);
    }
    getBackground().drawImage(currentDave, 0, getHeight() - 400);
}
```

7.2.3 Chuyển Cảnh (World Transition)

Khi biến đếm `count` đạt ngưỡng 500, lớp thực hiện giải phóng bộ nhớ bằng cách dừng âm thanh và sử dụng phương thức `Greenfoot.setWorld()` để chuyển sang `PlayScene`. Tại đây, toàn bộ đối tượng quản lý như `level` và `seedbank` sẽ được truyền đi dưới dạng tham số (*Dependency Injection*), giúp duy trì dữ liệu xuyên suốt các Scene.

8 SeedBank: Hệ Thống Quản Lý Thẻ Bài và Cơ Chế Kéo-Thả

8.1 Vai Trò và Chức Năng Chính

Lớp SeedBank đóng vai trò là trung tâm quản lý kinh tế và tương tác của người chơi. Nó không chỉ lưu trữ danh sách các gói hạt giống (SeedPacket) mà còn điều phối luồng logic từ lúc người chơi chọn thẻ cho đến khi thực thể cây được đặt xuống bàn chơi.

8.2 Thuật Toán Xử Lý Kéo-Thả (Drag-and-Drop Logic)

Đây là cơ chế phức tạp nhất của lớp, kết hợp giữa việc theo dõi chuột và tương tác với hệ thống lưới của GridManager. Quy trình được chia thành ba giai đoạn chính:

1. **Khởi tạo kéo (Detection):** Khi nhận sự kiện mousePressed, hệ thống sử dụng một hitbox để kiểm tra xem người chơi có chạm vào thẻ bài nào hợp lệ (đủ tiền và đã hồi chiêu) hay không.
2. **Phản hồi thị giác (Ghosting):** Một đối tượng tạm thời (TransparentObject) được tạo ra để đi theo con trỏ chuột. Nếu con trỏ nằm trong ô lưới có thể đặt (canPlace), "ảnh ma"này sẽ tự động bắt dính (*snapping*) vào tọa độ trung tâm của ô đó.
3. **Thực thi (Placement):** Khi nhả chuột (mouseDragEnded), hệ thống gọi phương thức placePlant() của GridManager. Nếu thành công, tài nguyên Sun sẽ bị trừ và thẻ bài bắt đầu quá trình hồi chiêu.

Listing 2: Logic bắt dính tọa độ khi kéo thẻ

```
if (gx >= 0 && gy >= 0 && GridManager.canPlace(gx, gy, plant)) {
    int tx = GridManager.getXCoord(gx, gy);
    int ty = GridManager.getYCoord(gx, gy);
    ghostImage.setLocation(tx, ty); // Snapping  h i u   ng
    ghostImage.setTransparent(true);
}
```

8.3 Quản Lý Tài Nguyên (Economy System)

SeedBank tích hợp lớp SunCounter để theo dõi lượng tài nguyên hiện có. Việc tách biệt logic hiển thị (SunCounter) và logic quản lý (SeedBank) giúp tuân thủ nguyên lý **Single Responsibility** trong OOP. Phương thức addSun() cho phép các thực thể khác (như AugmentCard hoặc Sun) cập nhật ngân sách của người chơi một cách an toàn.

8.4 Cập Nhật Cửa Hàng Động (Bank Refreshment)

Mỗi khi người chơi thực hiện "Roll"thẻ bài, phương thức updateBank() sẽ thực hiện dọn dẹp các thực thể cũ và tái cấu trúc danh sách thẻ mới.

- **Memory Management:** Tự động loại bỏ các thẻ cũ khỏi World để tránh rò rỉ bộ nhớ.
- **Augment Synergy:** Tích hợp biến isTDActive để cưỡng bức xuất hiện BonkchoyPacket nếu người chơi sở hữu lõi nâng cấp tương ứng.

8.5 Phản Hồi Trực Quan (Visual Feedback)

Lớp sử dụng tính năng `setTransparency(0)` cho chính nó để ẩn đi hình ảnh đại diện của đối tượng quản lý, chỉ tập trung hiển thị các thẻ bài con và số lượng Sun. Điều này giúp giao diện người dùng trông gọn gàng và chuyên nghiệp hơn theo các tiêu chuẩn UX hiện đại.

9 UpgradeManager: Quản Lý Logic Tiến Hóa Thực Thể

9.1 Kiến Trúc Lớp Tiện Ích (Utility Class Design)

`UpgradeManager` được thiết kế dưới dạng một lớp tĩnh (*Static Class*), không yêu cầu khởi tạo đối tượng. Việc sử dụng các phương thức `static` cho phép mọi thành phần trong hệ thống (như `GridManager` hoặc `PlayScene`) truy cập trực tiếp vào các quy tắc tiến hóa mà không làm phức tạp hóa vòng đời đối tượng.

9.2 Cơ Chế Phân Loại Thực Thể Bằng Polymorphism

Lớp sử dụng toán tử `instanceof` để xác định kiểu dữ liệu thực tế của thực thể tại thời điểm thực thi (*Run-time Type Identification*). Thuật toán chuyển đổi cấp độ được thiết lập theo mô hình tuyến tính, ánh xạ từ một thực thể cơ bản sang một thực thể nâng cấp có sức mạnh cao hơn.

- **Dòng Peashooter:** `Peashooter` → `Repeater` → `GatlingPea` → `GatlingPea2`.
- **Dòng Cactus:** `Cactus` → `Cactus2` → `Cactus3`.
- **Dòng BonkChoy:** `BonkChoy` → `BonkChoy2` → `BonkChoy3`.

9.3 Thuật Toán Quyết Định Kết Quả (Factory Method Pattern)

Phương thức `getUpgradeResult()` hoạt động tương tự như một **Factory Method** đơn giản. Nó đóng gói logic khởi tạo các lớp con cụ thể, giúp mã nguồn chính không cần biết chi tiết về quá trình tạo đối tượng mới:

Listing 3: Logic phân tầng cấp độ thực thể

```java
public static Plant getUpgradeResult(Plant p) {
    if (p instanceof Peashooter) return new Repeater();
    if (p instanceof Repeater) return new GatlingPea();
    // ... logic tiep tuc cho cac loai khac
    return null;
}
```

9.4 Khả Năng Mở Rộng và Bảo Trì

Dù cấu trúc hiện tại sử dụng chuỗi điều kiện `if-else`, thiết kế này cho phép dễ dàng tích hợp các cơ chế nâng cấp đặc biệt thông qua phương thức `getSpecialUpgrade()`.

- **Tách biệt logic (Decoupling):** Việc tách riêng logic nâng cấp ra khỏi lớp `Plant` giúp mã nguồn của các thực thể cây gọn gàng hơn, tập trung hoàn toàn vào hành vi chiến đấu (tấn công, hồi máu).
- **Dễ dàng cấu hình:** Khi muốn thay đổi thứ tự tiến hóa hoặc thêm cây mới, nhà phát triển chỉ cần cập nhật tại một vị trí duy nhất trong `UpgradeManager`.

9.5 Đánh Giá Kỹ Thuật

Việc sử dụng `instanceof` là một giải pháp trực quan và hiệu quả trong phạm vi framework Greenfoot cho các dự án game có quy mô vừa. Nó đảm bảo tính an toàn kiểu (*Type safety*) trước khi thực hiện các thao tác gộp thực thể trong `PlayScene`, ngăn chặn các lỗi khởi tạo đối tượng không mong muốn.

10 WaveManager: Hệ Thống Điều Phối Đợt Tấn Công và Sự Kiện

10.1 Vai Trò và Tầm Quan Trọng

`WaveManager` đóng vai trò là bộ máy vận hành chính (*Main Engine*) điều khiển tiến trình của trò chơi. Nó quản lý dòng thời gian, trạng thái của các đợt quái (*Waves*), và là nơi kích hoạt các cơ chế đặc biệt như lựa chọn lõi nâng cấp (*Augment Selection*) khi gặp các đợt tấn công lớn (*Huge Waves*).

10.2 Thuật Toán Quản Lý Thời Gian Thực (Delta Time)

Thay vì dựa vào tốc độ khung hình không ổn định của Greenfoot, `WaveManager` sử dụng `System.nanoTime()` để tính toán *Delta Time*. Điều này đảm bảo các sự kiện như sinh quái hoặc tặng Sun mỗi đợt diễn ra chính xác theo thời gian thực (miliseconds).

$$\Delta t = \frac{T_{current} - T_{last}}{1,000,000} \quad \text{(ms)} \tag{5}$$

10.3 Cơ Chế Phân Loại Đợt Tấn Công

Hệ thống quản lý đợt tấn công thông qua danh sách `hugeWaves`. Khi chỉ số `wave` trùng khớp với một phần tử trong danh sách này, trò chơi sẽ chuyển trạng thái từ chiến đấu sang chọn lõi (*Card Selection State*):

- **Normal Wave:** Gọi phương thức `sendWave()` để sinh quái ngay lập tức.
- **Huge Wave:** Kích hoạt `triggerCardSelection()`, tạm dừng tiến trình quái để người chơi nâng cấp sức mạnh.

10.4 Thuật Toán Sinh Quái và Định Vị Làn (Lane Positioning)

Việc sinh quái dựa trên dữ liệu ma trận 3 chiều được truyền từ `Arena`. `WaveManager` thực hiện ánh xạ từ chỉ số mảng sang tọa độ thực tế trên lưới lục giác thông qua `GridManager`:

Listing 4: Thuật toán ánh xạ tọa độ sinh quái

```
int row = board.clampRow(rowIndex);
int targetY = board.getYCoord(8, row) - 20; // Dieu chinh offset Y
int targetX = xOffset + (isHuge ? 100 : 0) + (j * 40); // Gian
    cach X
playScene.addObject(z, targetX, targetY);
```

10.5 Hệ Thống Phản Hồi và Thông Báo

Lớp này điều phối các thành phần UI để thông báo cho người chơi về tình trạng trận đấu:

- **ReadySetPlant:** Khởi tạo khi bắt đầu cấp độ.
- **WaveNotification:** Hiển thị thông báo khi bắt đầu một đợt quái lớn hoặc đợt quái cuối cùng (*Final Wave*).
- **Progress Bar:** Liên tục cập nhật tiến trình dựa trên tỉ lệ `wave / level.length`.

10.6 Điều Kiện Chiến Thắng (Win Condition)

Trạng thái chiến thắng được xác định dựa trên một logic kiểm tra kép:

$$\text{Victory} \iff (\text{wave} \geq \text{level.length}) \wedge (\text{count}(\text{Zombies}) = 0) \tag{6}$$

Điều này đảm bảo người chơi chỉ chiến thắng khi đã vượt qua đợt quái cuối cùng và tiêu diệt toàn bộ kẻ địch còn sót lại trên sân.

11 PlayScene

PlayScene đóng vai trò là **Controller trung tâm** (World) trong kiến trúc của trò chơi. Lớp này kế thừa từ greenfoot.World, chịu trách nhiệm điều phối toàn bộ vòng đời thực thi, quản lý trạng thái tài nguyên, và xử lý các tương tác giữa các hệ thống con (Sub-systems) như AI Zombie, Grid Logic và UI Hệ thống.

11.1 Thiết Kế Kiến Trúc Hệ Thống

11.1.1 Quản lý Thứ tự Hiển thị (Z-Order Management)

Để giải quyết vấn đề chồng lấp hình ảnh trong không gian 2D, PlayScene định nghĩa một phân cấp ưu tiên hiển thị thông qua phương thức setPaintOrder(). Thứ tự này đảm bảo các phản hồi trực quan quan trọng nhất luôn nằm ở lớp trên cùng.

Bảng 2: Phân tầng hiển thị trong PlayScene

Tầng (Layer)	Các thực thể (Actors)	Mô tả
Top Layer	AugmentCard, Transition	Giao diện lựa chọn và hiệu ứng chuyển cảnh.
UI Layer	SeedPacket, SunCounter	Các thành phần tương tác điều khiển.
Entity Layer	Plant, Zombie, Bullet	Các đối tượng tham gia chiến đấu chính.
Base Layer	GridManager, Background	Bản đồ lưới và hình ảnh nền.

11.1.2 Hệ thống Âm thanh Động (Dynamic Audio System)

Trò chơi áp dụng cơ chế thay đổi nhạc nền dựa trên cường độ đợt tấn công (*Wave intensity*). Thuật toán xác định nhạc nền có thể được mô tả qua hàm điều kiện:

$$f(\text{wave}) = \begin{cases} \text{EasyWaveMusic} & \text{nếu } 0 \leq \text{wave} \leq 6 \\ \text{FinalWaveMusic} & \text{nếu } 12 \leq \text{wave} \leq 16 \\ \text{Grasswalk (Standard)} & \text{các trường hợp còn lại} \end{cases} \tag{7}$$

11.2 Phân Tích Cơ Chế Core-Logic

11.2.1 Thuật toán Gộp Ba (Triple Merge Logic)

Đây là cơ chế cốt lõi được lấy cảm hứng từ thể loại *Auto-Chess*. Khi một thực thể Plant mới được khởi tạo, hệ thống sẽ thực hiện quét toàn bộ thế giới để tìm các thực thể cùng lớp (*same class*).

Listing 5: Cấu trúc logic kiểm tra Merge

```java
public void checkAndCombine(Class plantClass) {
    List<Plant> samePlants = getObjects(plantClass);
    // Loc nhung cay khong trong trang thai dang di chuyen gop
    List<Plant> available = samePlants.stream()
                                      .filter(p -> !p.isMerging)
                                      .collect(Collectors.toList()
                                          );

```

```
    if (available.size() >= 3) {
        executeMerge(available.subList(0, 3));
    }
}
```

11.2.2 Hệ thống Gacha và Xác suất Trọng số

Việc làm mới danh sách hạt giống (`rollPackets`) không sử dụng ngẫu nhiên thuần túy mà dựa trên **Weighted Random Selection** từ lớp `RupButton`. Điều này đảm bảo tính cân bằng game qua từng giai đoạn nâng cấp.

11.2.3 Điều kiện Thất bại (Lose Condition)

Hệ thống liên tục giám sát tọa độ x của tất cả thực thể `Zombie` trong phương thức `act()`. Điều kiện kết thúc trò chơi được xác định bởi:

$$\exists \mathcal{Z} \in \text{World} \mid X_{\mathcal{Z}} < \theta_{threshold} \tag{8}$$

Trong đó $\theta_{threshold} = 125$ (giới hạn an toàn của nhà chính).

11.3 Tối Ưu Hóa Hiệu Năng

`PlayScene` sử dụng `System.currentTimeMillis()` để quản lý các sự kiện thời gian thực (như tốc độ rơi của mặt trời) thay vì phụ thuộc vào `act()` cycle của Greenfoot. Điều này giúp:

- Đảm bảo tính ổn định của gameplay khi tốc độ mô phỏng thay đổi.
- Giảm thiểu gánh nặng tính toán cho CPU bằng cách tránh các vòng lặp kiểm tra trạng thái không cần thiết.

11.4 Kết Luận

Lớp `PlayScene` thể hiện một thiết kế hướng đối tượng chặt chẽ, đóng vai trò "nhạc trưởng"điều phối các luồng dữ liệu giữa UI, Logic chiến thuật và Hệ thống tài nguyên. Các cơ chế như Merge và Weighted Roll giúp tăng chiều sâu chiến thuật cho trò chơi *PvZ: Golden Spoon*.

12 RollButton

`RollButton` là một thành phần giao diện người dùng (UI) quan trọng trong hệ thống kinh tế của trò chơi, đóng vai trò là cơ chế **Gacha/Reroll**. Lớp này được xây dựng trên framework **Greenfoot (Java)** nhằm mục đích:

1. Quản lý chi phí tài nguyên (Sun) cho mỗi lượt làm mới cửa hàng.
2. Thực hiện thuật toán **Xác suất trọng số (Weighted Random)** để chọn vật phẩm.
3. Tương tác trực tiếp với hệ thống `SeedBank` để cập nhật danh sách thẻ bài.
4. Phản hồi trực quan và âm thanh cho hành động của người chơi.

12.1 Giới Thiệu

12.1.1 Vai trò trong Gameplay

Trong các trò chơi chiến thuật dạng *Auto Chess*, cơ chế "Roll"cho phép người chơi tiêu tốn tài nguyên để tìm kiếm các đơn vị mạnh hơn hoặc phù hợp với đội hình. `RollButton` thực hiện logic này bằng cách kết nối giữa `SeedBank` (nơi chứa thẻ) và `RupButton` (nơi quản lý tỉ lệ xuất hiện/Rarity).

12.1.2 Các thông số cốt lõi

Bảng 3: Thông số kỹ thuật của `RollButton`

Biến/Hằng số	Kiểu dữ liệu	Ý nghĩa
`ROLL_COST`	`int`	Chi phí cố định cho mỗi lượt roll (25 Sun).
`totalWeight`	`int`	Tổng trọng số của các loại thẻ trong bể tướng.
`currentPool`	`RarityEntry[]`	Danh sách các loại thẻ có thể xuất hiện theo cấp độ.

12.2 Thiết kế Giao diện (Rendering)

12.2.1 Kỹ thuật vẽ Graphics tầng thấp

Thay vì sử dụng hình ảnh có sẵn, `RollButton` sử dụng lớp `GreenfootImage` để vẽ trực tiếp bằng mã nguồn. Điều này giúp tối ưu hóa dung lượng và cho phép thay đổi nội dung động (ví dụ: hiển thị giá tiền).

```java
private void updateAppearance() {
    GreenfootImage bg = new GreenfootImage(100, 45);
    bg.setColor(new Color(0, 0, 0, 180)); // M u  n n  en  trong
        s u t
    bg.fill();

    bg.setColor(Color.WHITE);
    bg.drawRect(0, 0, 99, 44); //  V   v i n  n  t

    bg.setFont(new Font("Arial", true, false, 14));
    bg.drawString("ROLL SEEDS", 6, 20);

```

```
    bg.setColor(Color.YELLOW); // M u v ng c tr ng cho
        t i nguy n Sun
    bg.drawString("$" + ROLL_COST, 10, 38);

    setImage(bg);
}
```

12.3 Thuật toán Xác suất Trọng số (Weighted Random)

12.3.1 Nguyên lý hoạt động

Thuật toán chọn thẻ trong `executeRoll` dựa trên nguyên lý **Cursor Selection** (con trỏ tích lũy). Mỗi đơn vị có một trọng số (`weight`). Tổng các trọng số tạo thành một khoảng $[0, \text{totalWeight})$.

$$P(\text{card}_i) = \frac{\text{weight}_i}{\sum_{j=1}^{n} \text{weight}_j} \tag{9}$$

12.3.2 Cài đặt mã nguồn

Lớp sử dụng `getDeclaredConstructor().newInstance()` (Java Reflection) để khởi tạo các đối tượng thẻ từ lớp (`Class`) được định nghĩa trong `RarityEntry`.

```
int randomNumber = Greenfoot.getRandomNumber(totalWeight);
int cursor = 0;
for (RupButton.RarityEntry entry : currentPool) {
    cursor += entry.weight;
    if (randomNumber < cursor) {
        // K h i t o instance m i c a SeedPacket t class
            nh ngh a s n
        newBank[i] = (SeedPacket) entry.packetClass.
            getDeclaredConstructor().newInstance();
        break;
    }
}
```

12.4 Luồng xử lý sự kiện (Event Flow)

1. **Kiểm tra trạng thái:** Phương thức `act()` kiểm tra xem người chơi có đang trong trạng thái chọn thẻ (`choosingCard`) hay không để chặn click trùng lặp.
2. **Kiểm tra tài chính:** So sánh `world.seedbank.getSun()` với `ROLL_COST`.
3. **Trừ tài nguyên:** Nếu đủ điều kiện, thực hiện trừ Sun và phát âm thanh thông báo.
4. **Tạo danh sách mới:** Chạy vòng lặp 3 lần (tương ứng 3 ô card) để chọn ngẫu nhiên thẻ mới.
5. **Cập nhật UI:** Gọi `world.seedbank.updateBank(newBank)` để hiển thị thẻ lên màn hình.

12.5 Kết Luận

Lớp `RollButton` thể hiện một cách tiếp cận lập trình hướng đối tượng sạch sẽ (*clean code*) bằng cách phân tách rõ ràng giữa logic xử lý dữ liệu và logic hiển thị. Việc sử dụng **Reflection** giúp hệ thống linh hoạt, cho phép thêm loại thẻ mới mà không cần sửa đổi mã nguồn của nút Roll.

13 RupButton

`RupButton` (Rarity Upgrade Button) là lớp quản lý hệ thống cấp độ người chơi và kiểm soát tỷ lệ xuất hiện của các thẻ bài (*gacha rates*) trong game. Lớp này đảm nhận ba trách nhiệm chính:

1. Lưu trữ danh sách các thực thể bài (`RarityEntry`) và trọng số (`weight`) tương ứng.
2. Xử lý logic nâng cấp cấp độ (*Level-up system*) khi người chơi đủ tài nguyên.
3. Điều chỉnh động xác suất xuất hiện của thẻ bài dựa trên cấp độ hiện tại (*Dynamic Balancing*).

13.1 Cấu Trúc Dữ Liệu và Thành Phần

13.1.1 Lớp Nội Tại: `RarityEntry`

Để quản lý xác suất một cách linh hoạt, lớp sử dụng một `static class` phụ trợ để ánh xạ giữa lớp đối tượng thẻ bài và trọng số của nó.

```
public static class RarityEntry {
    public Class packetClass; // Class cua the bai (vd:
        PeashooterPacket)
    public int weight;        // Trong so xac suat
}
```

13.1.2 Cơ Chế "Bad Luck Protection"Ban Đầu

Một điểm đặc biệt trong thiết kế của `RupButton` là phương thức `getPoolForRoll()`. Trong 2 lượt roll đầu tiên (`rollCount < 2`), hệ thống cưỡng bức (*forced*) tỷ lệ xuất hiện của `PotatoPacket` lên 100%. Điều này giúp đảm bảo người chơi có khởi đầu thuận lợi (một dạng *scripted events* trong lập trình game).

13.2 Logic Nâng Cấp và Tiến Hóa Xác Suất

13.2.1 Hệ Thống Cấp Độ

Khi người chơi kích hoạt `executeUpgrade()`, hệ thống thực hiện kiểm tra điều kiện kép:

- **Giới hạn cấp độ:** `currentLevel < MAX_LEVEL` (tối đa cấp 5).
- **Khả năng tài chính:** `seedbank.getSun() >= UPGRADE`$_COST$.

13.2.2 Thuật Toán Thay Đổi Trọng Số (`applyChanges`)

Đây là "trái tim"của hệ thống cân bằng game. Khi cấp độ tăng lên, trọng số của các thẻ bài cũ giảm xuống hoặc biến mất, nhường chỗ cho các thẻ bài cao cấp hơn.

Công thức tính xác suất xuất hiện P của một thẻ bài i tại cấp độ L được xác định bởi:

$$P(i|L) = \frac{W_i(L)}{\sum_{j=1}^{n} W_j(L)} \tag{10}$$

Trong đó $W_i(L)$ là hàm trọng số của thẻ i tại cấp độ L.

Bảng 4: Sự thay đổi trọng số theo cấp độ (Ví dụ)

Thẻ Bài	Level 1	Level 4	Level 5
Peashooter	3	3	0 (Loại bỏ)
Repeater	0	2	7 (Chủ lực)
GatlingPea	0	1	3 (Siêu hiếm)
Cactus	3	0 (Loại bỏ)	0

13.3 Giao Diện và Phản Hồi Người Chơi

13.3.1 Trạng Thái Hiển Thị

Lớp `updateAppearance()` sử dụng các kỹ thuật đồ họa vector của Greenfoot để thay đổi trạng thái nút:

- **Đang hoạt động:** Hiển thị cấp độ hiện tại và giá tiền (`Color.YELLOW`).
- **Cấp tối đa:** Hiển thị trạng thái "COMPLETED"và đổi màu sang xám (`Color.GRAY`), tạo tín hiệu thị giác cho người chơi rằng không thể nâng cấp thêm.

13.4 Kết Luận

`RupButton` không chỉ là một nút bấm thông thường mà là một **Controller** điều phối độ khó của game. Việc tách biệt giữa dữ liệu trọng số (`weightedPool`) và logic áp dụng (`applyChanges`) cho phép nhà phát triển dễ dàng tinh chỉnh chỉ số (*game balancing*) mà không làm ảnh hưởng đến cấu trúc vận hành của hệ thống.

14 AugmentCard

14.1 Vai Trò trong Hệ Thống Gameplay

`AugmentCard` là thành phần giao diện tương tác xuất hiện giữa các giai đoạn chuyển đợt (*Wave Transition*). Lớp này thực hiện cơ chế lựa chọn "Lõi công nghệ"(Augments), cho phép người chơi nhận các lợi ích về kinh tế hoặc sức mạnh để tùy biến chiến thuật theo thời gian thực.

14.2 Thuật Toán Xử Lý Tương Tác Chuột (UX Animation)

Để cải thiện trải nghiệm người dùng (*User Experience*), lớp tích hợp thuật toán thay đổi kích thước động khi người chơi di chuyển chuột qua (*Hover effect*). Cơ chế này được tối ưu hóa để tránh hiện tượng nhấp nháy (*flickering*) bằng cách sử dụng biến trạng thái `hovered`.

Listing 6: Logic xử lý hiệu ứng Hover

```java
private void handleHover() {
    if (Greenfoot.mouseMoved(this)) {
        if (!hovered) {
            hovered = true;
            updateImage(W_HOVER, H_HOVER); // Phong to
        }
    }
    if (Greenfoot.mouseMoved(null) && !Greenfoot.mouseMoved(this))
        {
        if (hovered) {
            hovered = false;
            updateImage(W_NORMAL, H_NORMAL); // Thu nho ve mac
                dinh
        }
    }
}
```

Thuật toán này đảm bảo rằng việc thay đổi tài nguyên đồ họa (`scale image`) chỉ xảy ra duy nhất một lần tại thời điểm chuyển giao trạng thái (*State Transition*), giúp tối ưu hóa hiệu năng CPU.

14.3 Cơ Chế Áp Dụng Hiệu Ứng (Augment Logic)

Khi người chơi kích hoạt một thẻ bài, phương thức `applyAugmentEffect()` sẽ truy xuất vào hệ thống quản lý tài nguyên của `PlayScene`. Cấu trúc logic dựa trên định danh chuỗi (`String type`) để phân loại các loại nâng cấp:

- **Reroll Card:** Hỗ trợ tài chính trực tiếp để làm mới cửa hàng.
- **TD / HM:** Các lõi đặc thù cung cấp Sun (150 đơn vị) để tái thiết lập đội hình.

14.4 Quản Lý Luồng Sự Kiện (Event Flow)

Sau khi một lựa chọn được thực hiện, lớp thực thi một chuỗi các hành động dọn dẹp hệ thống (*UI Cleanup*):

1. **Xóa thực thể:** Loại bỏ toàn bộ các thẻ `AugmentCard` và lớp phủ `Overlay` khỏi thế giới thông qua phương thức `removeObjects()`.
2. **Tiếp diễn tiến trình:** Gọi hàm `manager.nextWave()` để kích hoạt đợt tấn công tiếp theo.

Điều này thể hiện mô hình lập trình hướng sự kiện (*Event-driven programming*), trong đó UI đóng vai trò là tác nhân kích hoạt các thay đổi trạng thái quan trọng trong bộ máy vận hành game (*Game Engine*).

15 PhysicsBody

`PhysicsBody` là một lớp trừu tượng (*abstract class*) được thiết kế để khắc phục hạn chế về hệ tọa độ số nguyên mặc định của framework **Greenfoot**. Lớp này đóng vai trò là nền tảng vật lý cho mọi đối tượng di động trong game, đảm nhận các nhiệm vụ:

1. Lưu trữ vị trí thực thể dưới dạng số thực (*double*) để đạt độ chính xác cực cao.
2. Xử lý chuyển động dựa trên các hàm lượng giác (`sin`, `cos`).
3. Khử hiện tượng rung lắc (*jittering*) khi đối tượng di chuyển ở vận tốc thấp.
4. Đồng bộ hóa tọa độ thực tế (*exact coordinates*) với tọa độ hiển thị (*render coordinates*).

15.1 Giới Thiệu

15.1.1 Bối Cảnh và Vấn Đề

Trong Greenfoot, lớp `Actor` sử dụng kiểu dữ liệu `int` để quản lý tọa độ x và y. Đối với các tựa game yêu cầu tính toán quỹ đạo tỉ mỉ như *PvZ: Thìa Vàng*, điều này dẫn đến sai số làm tròn (*rounding error*). Ví dụ: Nếu một Zombie có vận tốc 0.4 pixel/frame, hệ thống sẽ làm tròn về 0, khiến đối tượng không thể di chuyển.

`PhysicsBody` giải quyết vấn đề này bằng kỹ thuật **Sub-pixel Movement**, cho phép các thực thể tích lũy các khoảng cách di chuyển cực nhỏ qua từng frame cho đến khi đủ để thay đổi 1 đơn vị pixel trên màn hình.

15.1.2 Sơ đồ phân cấp lớp

Bảng 5: Các lớp kế thừa từ `PhysicsBody`

Lớp con	Lợi ích nhận được
`Zombie`	Di chuyển mượt mà ngay cả khi bị làm chậm (Slow).
`Projectile`	Bay theo đường chéo chính xác đến mục tiêu trên lưới lục giác.
`VFX_Effect`	Các hiệu ứng hạt (particles) di chuyển tự nhiên hơn.

15.2 Cấu Trúc Dữ Liệu và Định Nghĩa

15.2.1 Biến Thành Viên

```
private double exactX;
private double exactY;
```

Hai biến này lưu trữ vị trí thực của đối tượng trong không gian Euclid. Mọi tính toán logic đều dựa trên cặp giá trị này thay vì các hàm `getX()` hay `getY()` của lớp cha.

15.2.2 Cơ Chế Đồng Bộ Hóa (Overriding)

Để đảm bảo tính nhất quán, lớp ghi đè phương thức `setLocation` để cập nhật đồng thời cả tọa độ thực và tọa độ hiển thị:

```
public void setLocation(double x, double y) {
    exactX = x;
    exactY = y;
```

```
    // Thuat toan lam tron de hien thi tren pixel gan nhat
    super.setLocation((int) (x + 0.5), (int) (y + 0.5));
}
```

Việc sử dụng `(int) (x + 0.5)` là một kỹ thuật làm tròn nhanh (*Fast Rounding*) giúp đối tượng hiển thị tại vị trí pixel tối ưu nhất so với vị trí thực tế của nó.

15.3 Thuật Toán Di Chuyển Lượng Giác

15.3.1 Phân Rã Vector Chuyển Động

Khi gọi hàm `move(double distance)`, hệ thống sẽ phân rã khoảng cách di chuyển thành hai thành phần dx và dy dựa trên góc xoay hiện tại (θ) của đối tượng.
Công thức toán học áp dụng:

$$\theta_{rad} = \text{toRadians(Rotation)} \tag{11}$$
$$dx = \cos(\theta_{rad}) \cdot \text{distance} \tag{12}$$
$$dy = \sin(\theta_{rad}) \cdot \text{distance} \tag{13}$$

15.3.2 Cài Đặt Mã Nguồn

```
public void move(double distance) {
    double radians = Math.toRadians(getRotation());
    double dx = Math.cos(radians) * distance;
    double dy = Math.sin(radians) * distance;
    setLocation(exactX + dx, exactY + dy);
}
```

Ưu điểm: Phương thức này cho phép đối tượng di chuyển theo bất kỳ góc nào mà không bị bó buộc vào 8 hướng cơ bản, cực kỳ quan trọng cho các hiệu ứng đạn bay trong lưới lục giác.

15.4 Ứng Dụng Trong Logic Game

15.4.1 Xử lý va chạm chính xác

Nhờ có `getExactX()` và `getExactY()`, các hàm kiểm tra va chạm (*Collision Detection*) giữa đạn và Zombie có thể sử dụng công thức khoảng cách Euclid với độ chính xác cao hơn:

$$d = \sqrt{(x_{target} - x_{bullet})^2 + (y_{target} - y_{bullet})^2} \tag{14}$$

15.4.2 Khả năng mở rộng

Cấu trúc trừu tượng của `PhysicsBody` cho phép dễ dàng thêm các thuộc tính vật lý khác như gia tốc (*acceleration*), lực ma sát (*friction*) hoặc lực đẩy (*knockback*) mà không cần thay đổi cấu trúc của các lớp con như `Zombie`.

15.5 Kết Luận Sơ Bộ

Việc triển khai `PhysicsBody` là một bước đi tất yếu để nâng cấp dự án từ một ứng dụng Greenfoot cơ bản lên một sản phẩm game có độ hoàn thiện cao. Nó không chỉ giải quyết

triệt để vấn đề sai số hiển thị mà còn tạo tiền đề vững chắc cho việc xử lý các hiệu ứng vật lý phức tạp hơn trong tương lai.

16 SpriteAnimator

`SpriteAnimator` là lớp hỗ trợ trung gian, kế thừa từ `PhysicsBody`, chuyên trách quản lý hệ thống hoạt ảnh (*animation*) cho các thực thể. Lớp này giải quyết bài toán đồng bộ hóa thời gian thực trong Greenfoot bằng cách:

1. Tự động nhập và quản lý chuỗi hình ảnh (*Sprite Sheets*).
2. Sử dụng **Delta Time** để điều khiển tốc độ hoạt ảnh độc lập với FPS.
3. Cung cấp cơ chế *Hit-flash* (nháy sáng) khi thực thể nhận sát thương.
4. Hỗ trợ đa dạng các chế độ lặp (*Looping*) và tỷ lệ quy đổi (*Scaling*).

16.1 Cơ Chế Quản Lý Thời Gian (Delta Time)

16.1.1 Lý thuyết điều khiển khung hình

Trong Greenfoot, hàm `act()` chạy dựa trên tốc độ xử lý của CPU, dẫn đến việc hoạt ảnh có thể quá nhanh hoặc quá chậm tùy máy. `SpriteAnimator` khắc phục điều này bằng cách đo lường khoảng thời gian trôi qua giữa hai khung hình (Δt).

$$\Delta t = \frac{T_{current} - T_{last}}{10^6} \quad \text{(đơn vị: miliseconds)} \tag{15}$$

16.1.2 Cài đặt thuật toán Animation

Chỉ khi Δt vượt quá một ngưỡng thời gian (`duration`) định trước, chỉ số khung hình mới được tăng lên.

Listing 7: Thuật toán Animate dựa trên thời gian thực

```java
public void animate(GreenfootImage[] sprite, long duration) {
    currentFrame = System.nanoTime();
    deltaTime = (currentFrame - lastFrame) / 1000000;

    if (deltaTime > duration) {
        lastFrame = currentFrame;
        frame = (frame + 1) % sprite.length; // Quay vong khung
           hinh
        setImage(sprite[frame]);
    }
}
```

16.2 Xử Lý Tài Nguyên Hình Ảnh

16.2.1 Hệ thống nhập dữ liệu (Importing)

Lớp cung cấp các phương thức nạp chồng (*overloading*) để linh hoạt trong việc khởi tạo thực thể, hỗ trợ tự động thay đổi kích thước (*scaling*) để tiết kiệm tài nguyên bộ nhớ.

- `importSprites(String path, int count)`: Nạp chuỗi ảnh theo số lượng.
- `importSprites(..., double scaleFactor)`: Nạp ảnh kết hợp thay đổi tỷ lệ hiển thị.

16.2.2 Cơ chế Hit-Flash Effect

Đây là một biến tấu quan trọng để tăng tính phản hồi UI/UX. Khi thực thể trúng đòn, hệ thống sẽ tạm thời thay thế Sprite hiện tại bằng các khung hình "flash"(thường là màu trắng hoặc đỏ) thông qua một đối tượng `Timer`. Điều này tạo ra phản hồi thị giác ngay lập tức cho người chơi mà không làm gián đoạn luồng hoạt ảnh chính.

16.3 Ưu Điểm Kỹ Thuật

16.3.1 Tính tương thích và Hiệu năng

Bằng cách kế thừa từ `PhysicsBody`, `SpriteAnimator` thừa hưởng khả năng di chuyển chính xác cấp độ sub-pixel, đồng thời tách biệt hoàn toàn logic vật lý và logic hiển thị.

16.3.2 Khả năng tái sử dụng

Cấu trúc lớp trừu tượng cho phép mọi thực thể trong game (từ `Plant`, `Zombie` đến các hiệu ứng môi trường như `Dirt`) đều có thể sở hữu hoạt ảnh mượt mà chỉ với một dòng lệnh gọi hàm `animate()`.

16.4 Kết Luận Sơ Bộ

`SpriteAnimator` là giải pháp hoàn thiện cho vấn đề hiển thị trong Greenfoot. Việc áp dụng tính toán theo thời gian thực (*NanoTime*) thay vì đếm số lần gọi hàm `act()` giúp *PvZ: Thìa Vàng* duy trì được sự ổn định về mặt thị giác trên nhiều cấu hình máy tính khác nhau.

17 Plant

Lớp Plant là thành phần trung tâm trong hệ thống chiến đấu của dự án, kế thừa từ SpriteAnimator. Ngoài việc quản lý các thuộc tính sinh tồn cơ bản, lớp này tích hợp các cơ chế phức tạp đặc trưng của dòng game *Auto Chess*:

1. Quản lý vòng đời và tương tác vật lý (máu, sát thương, tử vong).
2. Hệ thống kéo-thả (*Drag and Drop*) tương tác với GridManager.
3. Cơ chế **Hợp nhất thực thể** (*Merging*) khi đủ 3 đơn vị cùng loại.
4. Điều phối chuyển động mượt mà thông qua lớp hỗ trợ Merger.

17.1 Cơ Chế Hợp Nhất (Merging System)

17.1.1 Lớp Hỗ Trợ Merger

Để tránh việc các đơn vị thay đổi vị trí tức thời gây mất thẩm mỹ, lớp Merger được thiết kế để điều khiển hành trình bay của các đơn vị nguyên liệu về phía đơn vị mục tiêu.

Listing 8: Thuật toán nội suy chuyển động trong Merger

```java
public boolean update() {
    double distance = Math.sqrt(dx * dx + dy * dy);
    if (distance < speed) {
        mover.setLocation(target.getX(), target.getY());
        return true; // Da den dich
    } else {
        double angle = Math.atan2(dy, dx);
        mover.setLocation(mover.getX() + (int)(Math.cos(angle) *
            speed),
                          mover.getY() + (int)(Math.sin(angle) *
                              speed));
        return false; // Dang di chuyen
    }
}
```

Thuật toán sử dụng hàm atan2 để xác định góc hướng chính xác theo radian, kết hợp với các hàm lượng giác để phân rã vận tốc theo hai trục X và Y.

17.1.2 Logic Nâng Cấp Thẻ Bài

Khi một đơn vị đạt đến điểm mục tiêu (onMergeReached), hệ thống thực hiện kiểm tra trạng thái toàn cục. Nếu tất cả các đơn vị nguyên liệu đã hội quân đủ, UpgradeManager sẽ được kích hoạt để thay thế thực thể cũ bằng một thực thể có cấp độ cao hơn (*Tier up*).

17.2 Xử Lý Tương Tác Người Dùng

17.2.1 Cơ Chế Kéo-Thả (Drag and Drop)

Hệ thống kéo-thả được cài đặt thông qua phương thức handleMouse(), cho phép người chơi tái cấu trúc đội hình ngay trong trận đấu.

- **Khởi đầu:** Ghi lại tọa độ lưới cũ (startGridX, startGridY).

- **Di chuyển:** Cập nhật vị trí Actor theo tọa độ chuột, đồng thời giảm độ đục (*transparency*) xuống 125 để tạo hiệu ứng thị giác.
- **Kết thúc:** Kiểm tra tính hợp lệ của ô đích thông qua `GridManager.placePlant()`. Nếu ô đích không hợp lệ hoặc nằm ngoài biên, thực hiện hàm `returnToOldPosition()`.

17.2.2 Giới Hạn Tương Tác

Để đảm bảo tính toàn vẹn dữ liệu, các thực thể đang trong trạng thái `isMerging` (đang bay về mục tiêu) hoặc `isTarget` (đang đợi được nâng cấp) sẽ bị khóa tương tác chuột để tránh xung đột logic.

17.3 Vòng Đời Thực Thể (Lifecycle)

17.3.1 Khởi Tạo và Đăng Ký Lưới

Khi một `Plant` được thêm vào `World`, nó tự động thực hiện quy trình đăng ký:

$$G_x = \text{GridManager.getGridX(initialX, initialY)} \tag{16}$$
$$G_y = \text{GridManager.getGridY(initialX, initialY)} \tag{17}$$

17.3.2 Quản Lý Tử Vong

Khi chỉ số `hp` (Health Points) về 0, phương thức `handleDeath()` được gọi để:

1. Giải phóng ô nhớ trong mảng `Board` của `GridManager`.
2. Phát âm thanh thông báo (*gulp.mp3*).
3. Gỡ bỏ đối tượng khỏi môi trường mô phỏng.

17.4 Kết Luận Sơ Bộ

Lớp `Plant` không chỉ là một đối tượng chứa dữ liệu mà là một **StateMachine** (máy trạng thái) quản lý linh hoạt giữa các chế độ: Đứng yên, Bị kéo, Di chuyển hợp nhất và Chờ nâng cấp. Việc tách biệt logic chuyển động vào lớp `Merger` giúp mã nguồn sạch sẽ và dễ bảo trì.

18 Peashooter

`Peashooter` là thực thể tấn công tầm xa cơ bản nhất trong *PvZ: Thìa Vàng*. Kế thừa từ lớp `Plant`, đơn vị này được tối ưu hóa về mặt hiệu năng thông qua việc lưu trữ tham chiếu thế giới (*Caching PlayScene*) và triển khai cơ chế cường hóa tạm thời (*Power-up Status*). Các chức năng chính bao gồm:

1. Tấn công mục tiêu trên hàng thông qua hệ thống `checkZombieInRow`.
2. Cơ chế **Plant Food**: Tăng tốc độ bắn đột biến trong thời gian ngắn.
3. Tối ưu hóa truy vấn bằng kỹ thuật *Caching* đối tượng thế giới.
4. Hệ thống hoạt ảnh chuyển đổi linh hoạt giữa các trạng thái `idle`, `shoot`, và cường hóa.

18.1 Cơ Chế Cường Hóa (Plant Food System)

18.1.1 Quản lý trạng thái Power-up

Khác với các đơn vị bắn thông thường, `Peashooter` tích hợp phương thức `activatePlantFood()` để thay đổi thông số chiến đấu ngay lập tức. Hệ thống sử dụng một bộ đếm thời gian thực để tự động hồi phục về trạng thái cơ bản.

- **Thời gian hiệu lực (`POWER_UP_DURATION`):** 3000ms.
- **Tốc độ bắn (`shootDelay`):** Giảm từ 1500ms xuống còn 300ms.

$$f_{attack} = \begin{cases} 3.33 \text{ shots/sec} & \text{khi } isPoweredUp = true \\ 0.67 \text{ shots/sec} & \text{khi } isPoweredUp = false \end{cases} \tag{18}$$

18.2 Tối Ưu Hóa Hiệu Năng

18.2.1 Kỹ thuật Caching PlayScene

Thay vì gọi hàm `(PlayScene) getWorld()` liên tục trong mỗi chu kỳ `update()` (một thao tác ép kiểu tốn kém tài nguyên), thực thể thực hiện lưu trữ tham chiếu ngay khi được thêm vào thế giới (`addedToWorld`):

Listing 9: Lưu trữ tham chiếu PlayScene để tối ưu tốc độ

```java
@Override
public void addedToWorld(World world) {
    super.addedToWorld(world);
    if (world instanceof PlayScene) {
        this.cachedPlayScene = (PlayScene) world;
    }
}
```

18.2.2 Xử lý luồng hành động (Action Handling)

Hàm `handleAction()` được thiết kế theo cấu trúc rẽ nhánh ưu tiên, đảm bảo hoạt ảnh `shoot` luôn mượt mà ngay cả khi nhịp độ bắn thay đổi nhanh chóng do cường hóa:

- **Độ trễ hoạt ảnh:** Được điều chỉnh động (`50ms` khi cường hóa so với `100ms` bình thường) để khớp với tốc độ ra đạn.
- **Trigger sinh đạn:** Được đặt tại `frame >= 1` để khớp với hình ảnh hạt đậu bay ra từ miệng cây trên Sprite.

18.3 Logic Kiểm Tra Mục Tiêu Hiện Đại

Thay vì sử dụng vòng lặp `for` truyền thống, `Peashooter` sử dụng **Java Stream API** để kiểm tra sự tồn tại của Zombie trong hàng một cách ngắn gọn và hiệu quả:

```
shooting = rowZombies.stream().anyMatch(z ->
    z.getWorld() != null &&
    z.getX() > getX() &&
    z.getX() <= cachedPlayScene.getWidth() + 10
);
```

Cách tiếp cận này không chỉ giúp mã nguồn dễ đọc hơn mà còn tận dụng tốt các tối ưu hóa nội tại của JVM khi xử lý danh sách đối tượng.

18.4 Kết Luận Sơ Bộ

`Peashooter` là minh chứng cho việc áp dụng các kỹ thuật lập trình Java hiện đại (*Streams, Caching*) vào môi trường Greenfoot. Việc triển khai cơ chế *Plant Food* cho thấy sự linh hoạt trong thiết kế máy trạng thái, cho phép thực thể thay đổi hành vi phức tạp mà vẫn duy trì được hiệu suất ổn định.

19 Repeater

Repeater là một biến thể nâng cao của đơn vị bắn đậu, được thiết kế để tăng gấp đôi hỏa lực trên một hàng lưới. Kế thừa từ lớp Plant, Repeater tích hợp các cơ chế điều khiển trạng thái phức tạp nhằm tối ưu hóa nhịp độ tấn công:

1. Hệ thống bắn đạn kép dựa trên biến đếm (shootCount).
2. Cơ chế kiểm tra kẻ thù theo thời gian thực trên cùng hàng (zombieRow).
3. Quản lý hoạt ảnh động giữa hai trạng thái: Nghỉ (*Idle*) và Bắn (*Shoot*).
4. Tích hợp hiệu ứng phản hồi *Hit-flash* riêng biệt cho từng trạng thái hoạt ảnh.

19.1 Logic Điều Khiển Nhịp Độ Tấn Công

19.1.1 Cơ chế bắn đạn kép (Burst Fire)

Khác với các đơn vị bắn đơn, Repeater sử dụng biến shootDelay kết hợp với nanoTime để tạo ra khoảng nghỉ giữa các đợt bắn, đồng thời sử dụng hoạt ảnh để kích hoạt hai lần bắn liên tiếp trong một chu kỳ tấn công.

$$\Delta t = \frac{T_{current} - T_{last}}{10^6} \geq \text{shootDelay} \tag{19}$$

Khi điều kiện thời gian thỏa mãn, thực thể chuyển sang trạng thái hoạt ảnh bắn. Đạn Pea được khởi tạo tại hai thời điểm khác nhau trong chuỗi khung hình hoạt ảnh, tạo ra hiệu ứng bắn hai viên đậu liên tiếp.

19.1.2 Thuật toán Nhận diện Kẻ thù

Để tối ưu hiệu năng, Repeater không tự tìm kiếm va chạm mà truy vấn trực tiếp danh sách kẻ thù từ PlayScene.

Listing 10: Logic quét mục tiêu trên hàng

```
for (Zombie i : world.level.zombieRow.get(getYPos())) {
    if (i != null && i.getX() > getX() && i.getX() <= world.
       getWidth()) {
        foundZombie = true; // Kich hoat trang thai shooting
        break;
    }
}
```

19.2 Quản Lý Hoạt Ảnh và Trạng Thái

19.2.1 Sự chuyển đổi trạng thái (Animation Switching)

Thực thể sử dụng một máy trạng thái logic để quyết định tập hợp hình ảnh hiển thị. Nếu không tìm thấy mục tiêu hoặc đang trong thời gian hồi chiêu (*Cooldown*), thực thể sẽ duy trì hoạt ảnh idle. Ngay khi mục tiêu xuất hiện, bộ hoạt ảnh shoot với tốc độ khung hình cao hơn (70ms/frame) sẽ được kích hoạt.

19.2.2 Phản hồi tương tác (Hit-flash Overriding)

Để đảm bảo tính nhất quán về thị giác khi thực thể chịu sát thương, phương thức `hit()` đã được ghi đè để nhận diện bộ Sprite hiện tại đang được sử dụng:

```
public void hit(int dmg) {
    if (!shootOnce) {
        hitFlash(idle, "repeater");
    } else {
        hitFlash(shoot, "repeatershoot");
    }
    hp -= dmg;
}
```

Điều này giúp hiệu ứng nháy sáng (*Flash*) luôn khớp với tư thế hiện tại của cây, tăng độ chân thực cho tương tác vật lý.

19.3 Kết Luận Sơ Bộ

Lớp `Repeater` thể hiện kỹ thuật quản lý thời gian chính xác trong lập trình game. Việc phối hợp giữa `deltaTime` và chỉ số khung hình hoạt ảnh (`frame index`) để kích hoạt sự kiện sinh đối tượng (*Spawn projectile*) là một giải pháp hiệu quả cho các nhân vật có hành vi tấn công theo chuỗi.

20 GatlingPea

GatlingPea là đơn vị hỏa lực hạng nặng được nâng cấp từ Repeater, kế thừa từ lớp Plant. Thực thể này được thiết kế để tối ưu hóa khả năng tấn công dồn dập thông qua cơ chế bắn chuỗi (*Burst-fire*), cho phép giải phóng một lượng lớn đạn đạo trong thời gian ngắn. Các đặc tính kỹ thuật bao gồm:

1. Hệ thống bắn chuỗi 7 viên đạn đậu (*7-shot burst*) trong một chu kỳ.
2. Cơ chế quản lý thời gian hồi (*Cooldown*) giữa các đợt bắn dựa trên nanoTime.
3. Tối ưu hóa hoạt ảnh tấn công với tốc độ khung hình cao (30ms/frame).
4. Logic quét mục tiêu theo hàng (*Row-scanning*) thời gian thực.

20.1 Cơ Chế Bắn Chuỗi Tốc Độ Cao

20.1.1 Thuật toán điều khiển chu kỳ đạn

Khác với các đơn vị bắn thông thường, GatlingPea vận hành theo cơ chế tích lũy số lượng đạn bắn ra (shootCount). Khi đạt ngưỡng 7 viên đạn, thực thể sẽ bắt buộc chuyển sang trạng thái chờ (Cooldown) để cân bằng lối chơi.

$$T_{action} = \begin{cases} \text{Animate(Shoot)} & \text{nếu } shootCount < 7 \\ \text{Animate(Idle)} & \text{nếu } shootCount \geq 7 \text{ và } \Delta t < 800ms \end{cases} \tag{20}$$

Việc sử dụng shootDelay = 800L tạo ra một khoảng nghỉ ngắn giữa các đợt bắn, mô phỏng lại nhịp độ nạp đạn của vũ khí hạng nặng.

20.1.2 Kỹ thuật Reset khung hình (Frame Manipulation)

Để đạt được tốc độ bắn cực nhanh, lớp thực hiện ghi đè biến frame ngay lập tức sau khi một viên đạn được sinh ra:

Listing 11: Logic bắn và reset khung hình tức thời

```
if (frame >= 4) {
    // Sinh dan tai toa do mom sung
    world.addObject(new Pea(getYPos()), getX() + 25, getY() - 17);
    frame = 0; // Reset ve khung hinh dau tien ngay lap tu
    shootCount++;
}
```

Kỹ thuật này cho phép loại bỏ các khung hình thừa trong chuỗi hoạt ảnh 19 frame, ép thực thể thực hiện vòng lặp "Chuẩn bị - Bắn"liên tục cho đến khi kết thúc chuỗi 7 viên.

20.2 Xử Lý Phản Hồi Tương Tác

20.2.1 Hệ thống Hit-flash động

Để duy trì tính nhất quán về mặt thị giác, phương thức hit() được lập trình để nhận diện trạng thái hành động của thực thể. Nếu GatlingPea đang trong chu kỳ bắn (shooting), hiệu ứng nháy sáng sẽ áp dụng trên bộ Sprite shoot, ngược lại sẽ áp dụng trên bộ idle.

20.2.2 Logic nhận diện mục tiêu (`checkForZombies`)

Thực thể thực hiện quét danh sách `zombieRow` để xác định sự hiện diện của kẻ thù. Điều kiện kích hoạt trạng thái tấn công yêu cầu Zombie phải thỏa mãn đồng thời:

- Đang tồn tại trong cùng một thế giới (`getWorld() != null`).
- Vị trí X nằm phía trước cây (`z.getX() > getX()`).
- Nằm trong phạm vi hiển thị của màn hình (`world.getWidth() + 10`).

20.3 Kết Luận Sơ Bộ

Lớp `GatlingPea` thể hiện sự kết hợp hiệu quả giữa việc điều khiển thời gian thực và thao tác trực tiếp trên biến trạng thái của hoạt ảnh. Việc tối ưu hóa chu kỳ bắn bằng cách reset `frame` thủ công là một giải pháp thông minh để vượt qua giới hạn về tốc độ hoạt ảnh mặc định, mang lại trải nghiệm chiến đấu mạnh mẽ cho người chơi.

21 GatlingPea2

GatlingPea2 đại diện cho đỉnh cao hỏa lực tầm xa trong dự án, kế thừa từ lớp Plant. Thực thể này được thiết kế với cơ chế tấn công theo đợt (*Burst-fire mechanism*) và khả năng cường hóa tối đa thông qua *Plant Food*. Các đặc tính kỹ thuật nổi bật gồm:

1. Hệ thống bắn chuỗi 5 viên đạn (*5-shot burst*) trong một chu kỳ tấn công.
2. Tích hợp loại đạn đặc biệt FirePea để gia tăng sát thương cơ bản.
3. Chế độ cường hóa *Overdrive*: Loại bỏ thời gian chờ giữa các đợt bắn và tăng tốc độ hoạt ảnh lên mức tối đa.
4. Logic điều khiển khung hình phức tạp để đồng bộ giữa âm thanh, hình ảnh và sự kiện sinh vật thể.

21.1 Cơ Chế Tấn Công Theo Đợt (Burst Logic)

21.1.1 Điều khiển chu kỳ bắn

Khác với Repeater bắn đạn kép, GatlingPea2 sử dụng biến shootCount để quản lý một đợt bắn gồm 5 lần ra đạn liên tiếp trước khi chuyển sang trạng thái hồi chiêu (*Cooldown*).

$$State = \begin{cases} \text{Shooting} & \text{nếu } shootCount < 5 \\ \text{Cooldown} & \text{nếu } shootCount \geq 5 \text{ và } \Delta t < shootDelay \\ \text{Reset} & \text{nếu } \Delta t \geq shootDelay \end{cases} \tag{21}$$

Việc áp dụng System.nanoTime() để tính toán deltaTime2 giúp nhịp độ hồi chiêu giữa các đợt bắn (2000ms) luôn chính xác, đảm bảo tính cân bằng cho trò chơi.

21.2 Cơ Chế Cường Hóa Plant Food

21.2.1 Trạng thái Overdrive

Khi kích hoạt activatePlantFood(), GatlingPea2 chuyển sang trạng thái cường hóa với các thông số được đẩy lên giới hạn vật lý của Engine:

- **Tốc độ hoạt ảnh:** Giảm từ 15ms xuống 10ms mỗi khung hình.
- **Độ trễ giữa các đợt bắn:** Giảm từ 2000ms xuống còn 40ms (gần như tức thời).
- **Hệ quả:** Tạo ra một "cơn mưa đạn"FirePea liên tục trong vòng 3 giây, có khả năng tiêu diệt hầu hết các loại Zombie mạnh nhất.

Listing 12: Logic xử lý bắn đạn liên hoàn

```java
private void executeShootingLogic() {
    int animSpeed = isPoweredUp ? 10 : 15;
    animate(shoot, animSpeed, false);

    if (frame >= 7) { // Diem kich hoat ban tren Sprite
        PlayScene.addObject(new FirePea(myRow), getX() + 25, getY
            () - 17);
        setFrame(1); // Reset ngay lap tu de ban vien tiep theo
        shootCount++;
    }
}
```

21.3 Phân Tích Hiệu Năng và Đồ Họa

21.3.1 Sự đồng bộ giữa Sprite và Sự kiện

Với bộ Sprite lên tới 19 khung hình (`GatlingPea`), việc chọn `frame >= 7` làm điểm kích hoạt đạn giúp khớp chính xác với hình ảnh nòng súng Gatling rực sáng. Việc gọi `setFrame(1)` ngay sau khi bắn cho phép thực thể bỏ qua các khung hình thừa, tập trung vào việc duy trì mật độ đạn cao.

21.3.2 Quản lý bộ nhớ

Mặc dù tần suất sinh đối tượng `FirePea` là rất lớn (đặc biệt khi cường hóa), nhờ vào cơ chế tự giải phóng (*Auto-removal*) đã thiết lập ở lớp `Projectile`, hiệu năng của hệ thống vẫn được duy trì ổn định mà không gây ra hiện tượng tràn bộ nhớ.

21.4 Kết Luận Sơ Bộ

`GatlingPea2` là minh chứng cho khả năng tùy biến sâu từ lớp cơ sở `Plant`. Việc kết hợp giữa logic toán học (đo thời gian nano), cấu trúc dữ liệu (danh sách Zombie theo hàng) và xử lý đồ họa (Sprite Animation) đã tạo ra một thực thể có hành vi phức tạp, mang lại cảm giác phấn khích cho người chơi trong các giai đoạn cao trào của trận đấu.

22 Cactus

Cactus (Xương rồng) là một thực thể tấn công tầm xa chuyên dụng, kế thừa từ lớp Plant. Với vai trò là một xạ thủ bắn tỉa, Cactus sở hữu lượng máu cơ bản cao hơn các đơn vị bắn đậu (100 HP) và cơ chế kiểm soát nhịp độ bắn nghiêm ngặt. Các đặc tính kỹ thuật chính bao gồm:

1. Tốc độ bắn chậm có chiến thuật với thời gian hồi (*Cooldown*) lên đến 2400ms.
2. Sử dụng loại đạn Needle với quỹ đạo bay ổn định.
3. Hệ thống hoạt ảnh chuyển đổi giữa trạng thái nghỉ (*Idle*) và trạng thái bắn (*Shoot*).
4. Khả năng quét mục tiêu chính xác trong hàng thông qua dữ liệu zombieRow.

22.1 Logic Điều Khiển Nhịp Độ Chiến Đấu

22.1.1 Quản lý thời gian nạp đạn (Reloading)

Để cân bằng với sức mạnh của loại đạn gai (Needle), Cactus được cấu hình với độ trễ bắn lớn (shootDelay = 2400L). Hệ thống sử dụng System.nanoTime() để tính toán khoảng delta thời gian chính xác, đảm bảo không có hiện tượng bắn lỗi khi tốc độ khung hình biến động.

$$\Delta t_{reload} = \frac{T_{now} - T_{lastShot}}{10^6} \geq 2400\,ms \tag{22}$$

Chỉ khi Δt_{reload} thỏa mãn, thực thể mới chuyển sang trạng thái hoạt ảnh bắn. Điều này tạo ra nhịp độ chiến đấu đặc trưng của một đơn vị bắn tỉa hạng nặng.

22.2 Xử Lý Hoạt Ảnh và Phản Hồi

22.2.1 Đồng bộ hóa khung hình và Sự kiện bắn

Quy trình thực thi một phát bắn được chia thành hai giai đoạn dựa trên biến cờ shootOnce:

- **Khởi tạo:** Khi đủ thời gian hồi, frame được đưa về 0 để bắt đầu chuỗi hoạt ảnh shoot.
- **Kích hoạt:** Đạn Needle chỉ được sinh ra khi hoạt ảnh đạt tới frame >= 2. Ngay sau đó, lastFrame2 được cập nhật để bắt đầu chu kỳ hồi chiêu mới.

Listing 13: Logic sinh đạn đồng bộ với hoạt ảnh

```
if (frame >= 2 && getWorld() != null) {
    // Sinh dan tai vi tri mieng xuong rong
    PlayScene.addObject(new Needle(getYPos()), getX() + 30, getY()
        - 8);
    lastFrame2 = currentFrame; // Bat dau dem thoi gian hoi
}
```

22.2.2 Hệ thống Hit-flash thích ứng

Phương thức hit() được lập trình để phản hồi trực quan ngay lập tức khi Cactus chịu sát thương. Nhờ vào việc kiểm tra trạng thái shootOnce, hiệu ứng nháy sáng (*Flash*) luôn được áp dụng chính xác lên bộ ảnh mà nhân vật đang hiển thị, đảm bảo không xảy ra hiện tượng lỗi hình ảnh (*Sprite glitch*).

22.3 Thuật Toán Nhận Diện Kẻ Thù

`Cactus` duy trì một bộ lọc mục tiêu thông minh. Nó chỉ kích hoạt trạng thái `shooting` khi thỏa mãn ba điều kiện:

1. Danh sách `zombieRow` tại hàng tương ứng không trống.
2. Thực thể Zombie mục tiêu đang tồn tại hợp lệ trong thế giới.
3. Vị trí X của Zombie nằm trong phạm vi từ vị trí của cây đến biên phải màn hình cộng thêm biên an toàn (`PlayScene.getWidth() + 10`).

22.4 Kết Luận Sơ Bộ

Lớp `Cactus` là minh chứng cho việc thiết kế một đơn vị có nhịp độ chậm nhưng hiệu quả cao. Việc kiểm soát chặt chẽ mối quan hệ giữa `shootDelay` và hoạt ảnh giúp thực thể này vận hành ổn định, đóng vai trò quan trọng trong việc tiêu diệt các mục tiêu đơn lẻ bền bỉ trong *PvZ: Thìa Vàng*.

23 Cactus2

Cactus2 là biến thể nâng cấp cấp độ 2 của thực thể xương rồng. Lớp này kế thừa cấu trúc từ Plant nhưng được tái thiết kế logic chiến đấu để chuyển đổi từ một xạ thủ bắn đơn sang một đơn vị kiểm soát vùng với khả năng bắn đa hướng. Các đặc tính nổi bật bao gồm:

1. Cơ chế bắn đạn kép với quỹ đạo lệch tâm (*Offset Projectiles*).
2. Hệ thống cường hóa *Plant Food* cho phép giảm 80% thời gian hồi chiêu.
3. Áp dụng kỹ thuật tối ưu hóa *Caching* đối tượng PlayScene.
4. Sử dụng Needle2 - loại đạn xuyên thấu có khả năng di chuyển theo trục dọc.

23.1 Cơ Chế Bắn Đa Hướng (Multi-directional Logic)

23.1.1 Thuật toán Phân rã Đạn đạo

Khác với các đơn vị bắn thẳng thông thường, Cactus2 triển khai cơ chế bắn hai viên đạn đồng thời hướng về các khu vực lân cận của hàng mục tiêu. Điều này được thực hiện thông qua tham số offset trong phương thức executeShoot().

Listing 14: Logic khởi tạo đạn đạo đa hướng

```
private void executeShoot() {
    int offset = 50; // Do lech toa do Y
    // Ban vien dan cheo len tren
    getWorld().addObject(new Needle2(getY(), -offset), getX(),
        getY());
    // Ban vien dan cheo xuong duoi
    getWorld().addObject(new Needle2(getY(), offset), getX(), getY
        ());

    lastFrame2 = currentFrame;
    shootOnce = false;
}
```

Nhờ sử dụng lớp đạn Needle2, các viên đạn này không chỉ bay thẳng mà còn tự động điều chỉnh tọa độ Y để bao phủ một phạm vi rộng hơn trên chiến trường, tăng xác suất trúng mục tiêu trong lưới lục giác.

23.2 Hệ Thống Cường Hóa và Quản Lý Trạng Thái

23.2.1 Cơ chế Plant Food (Overdrive)

Khi kích hoạt trạng thái cường hóa, Cactus2 thay đổi hoàn toàn nhịp độ tấn công. Biến shootDelay được giảm từ 1500ms xuống còn 300ms, đồng thời tốc độ hoạt ảnh được đẩy lên mức tối đa để khớp với tần suất ra đạn.

$$T_{action} = \begin{cases} 300ms & \text{nếu } isPoweredUp = true \\ 1500ms & \text{nếu } isPoweredUp = false \end{cases} \tag{23}$$

Trạng thái này được duy trì trong 3 giây (POWER_UP_DURATION) nhờ vào sự phối hợp giữa System.currentTimeMillis() và phương thức updatePowerUpStatus().

23.3 Tối Ưu Hóa Hệ Thống

23.3.1 Caching và Hiệu suất truy vấn

Để giảm thiểu chi phí tính toán trong mỗi vòng lặp `act()`, `Cactus2` thực hiện lưu trữ tham chiếu đến `PlayScene` ngay khi được khởi tạo. Đồng thời, việc kiểm tra Zombie trong hàng được thực hiện bằng **Java Stream API**, giúp mã nguồn tối giản và đạt hiệu năng cao:

```
shooting = rowZombies.stream().anyMatch(z ->
    z.getWorld() != null &&
    z.getX() > getX() &&
    z.getX() <= cachedPlayScene.getWidth() + 10
);
```

23.4 Kết Luận Sơ Bộ

`Cactus2` thể hiện sự nâng cấp toàn diện về mặt logic so với phiên bản tiền nhiệm. Việc kết hợp giữa đạn xuyên thấu đa hướng và cơ chế cường hóa biến thực thể này thành một công cụ phòng thủ chiến lược, có khả năng dọn dẹp hàng ngũ kẻ thù trên diện rộng chỉ trong thời gian ngắn.

24 Cactus3

Cactus3 là cấp độ tiến hóa cuối cùng của thực thể xương rồng trong hệ thống phòng thủ. Biến thể Tier 3 này được thiết kế để tối ưu hóa khả năng chế áp đa mục tiêu thông qua việc triển khai hệ thống đạn đạo "Quạt"(Spread-shot). Các cải tiến vượt bậc bao gồm:

1. Khả năng bắn đồng thời 3 luồng đạn đạo xuyên thấu (Needle3) theo các hướng khác nhau.
2. Tầm phủ sóng bao quát hàng hiện tại và hai hàng lân cận (trên và dưới).
3. Tích hợp cơ chế *Plant Food* cường hóa tốc độ bắn lên mức 3.33 phát/giây.
4. Sử dụng thuật toán anyMatch của Java Stream để tối ưu hóa việc quét mục tiêu trên phạm vi rộng.

24.1 Cơ Chế Đạn Đạo Hình Quạt (Spread-shot Mechanism)

24.2 Thuật toán Phân rã 3 tia

Điểm đột phá của Cactus3 nằm ở phương thức executeShoot(). Thay vì bắn thẳng hoặc bắn đôi, lớp này khởi tạo 3 đối tượng Needle3 với các tham số độ lệch (*offset*) khác nhau, mô phỏng quỹ đạo hình quạt.

Listing 15: Logic khởi tạo 3 luồng đạn đạo song song

```
private void executeShoot() {
    // Tia 1: Lech len hang tren (offset -50)
    getWorld().addObject(new Needle3(getY(), -50), getX(), getY())
        ;

    // Tia 2: Ban thang tai hang hien tai (offset 0)
    getWorld().addObject(new Needle3(getY(), 0), getX(), getY());

    // Tia 3: Lech xuong hang duoi (offset 50)
    getWorld().addObject(new Needle3(getY(), 50), getX(), getY());

    lastFrame2 = currentFrame;
    shootOnce = false;
}
```

Nhờ vào đặc tính của lớp Needle3, các viên đạn này sẽ tự động di chuyển về phía tọa độ Y mục tiêu trong khi vẫn tiến về phía trước, tạo ra một lưới đạn dày đặc không thể xuyên thủng.

24.3 Cường Hóa Trạng Thái Tối Thượng

24.3.1 Hiệu ứng Plant Food (Berserk Mode)

Khi nhận được thức ăn cho cây, Cactus3 giải phóng toàn bộ tiềm năng chiến đấu. Hệ thống ghi đè các tham số hoạt ảnh và thời gian hồi chiêu để đạt được mật độ hỏa lực cực đại:

- **Tốc độ bắn:** Rút ngắn từ 1500ms xuống còn 300ms.
- **Tốc độ hoạt ảnh:** Đạt mức 2ms mỗi khung hình (gần như tức thời).

- **Mật độ đạn:** Tổng cộng 30 viên đạn xuyên thấu được giải phóng chỉ trong vòng 3 giây cường hóa.

$$\text{Total Projectiles} = \frac{T_{duration}}{T_{delay}} \times N_{lanes} = \frac{3000}{300} \times 3 = 30 \tag{24}$$

24.4 Tối Ưu Hóa Truy Vấn và Hiệu Năng

24.4.1 Kỹ thuật Caching và Java Stream

Dù xử lý lượng đạn đạo lớn, `Cactus3` vẫn duy trì hiệu suất ổn định nhờ vào việc **Caching PlayScene** và áp dụng Functional Programming trong Java để kiểm tra điều kiện bắn:

```
shooting = rowZombies.stream().anyMatch(z ->
    z.getWorld() != null &&
    z.getX() > getX() &&
    z.getX() <= cachedPlayScene.getWidth() + 10
);
```

Việc sử dụng `anyMatch` cho phép dừng duyệt danh sách ngay khi tìm thấy Zombie đầu tiên trong phạm vi, giảm thiểu độ phức tạp thuật toán $O(n)$ trong các tình huống hàng đợi Zombie quá dài.

24.5 Kết Luận Sơ Bộ

`Cactus3` là biểu tượng cho sức mạnh phòng thủ diện rộng trong *PvZ: Thìa Vàng*. Bằng cách kết hợp giữa logic đạn đạo đa mục tiêu và các kỹ thuật lập trình Java hiện đại, lớp này cung cấp một giải pháp chiến thuật tối ưu, cho phép người chơi kiểm soát chiến trường một cách chủ động và mạnh mẽ.

25 BonkChoy1

BonkChoy là một đơn vị cận chiến đặc thù trong hệ thống phòng thủ, kế thừa từ lớp Plant. Khác với các đơn vị tầm xa, BonkChoy tập trung vào logic tấn công liên hoàn (*Combo System*) và khả năng tự hồi phục dựa trên hành động tấn công. Các đặc điểm chính bao gồm:

1. Hệ thống hoạt ảnh phân tầng: Nghỉ (*Idle*), Tấn công thường (*Punch*) và Tấn công kết liễu (*Knockout*).
2. Logic tính toán sát thương dựa trên trạng thái sinh tồn (Sát thương tăng khi đang bị tấn công).
3. Cơ chế hồi phục máu (*Healing*) sau mỗi chu kỳ đòn đánh.
4. Tối ưu hóa tài nguyên thông qua phương thức tiền xử lý ảnh loadResized.

25.1 Cấu Trúc Hoạt Ảnh và Tiền Xử Lý

25.1.1 Kỹ thuật Tối ưu hóa Bộ nhớ (Resizing)

Để đảm bảo hiệu năng và tính thẩm mỹ, BonkChoy thực hiện thu nhỏ kích thước Sprite ngay từ bước khởi tạo thay vì thay đổi trong mỗi chu kỳ act().

$$Size_{new} = Size_{old} \times 0.45 \tag{25}$$

Việc xử lý này giúp giảm tải cho bộ nhớ RAM của Greenfoot khi phải lưu trữ hàng trăm khung hình hoạt ảnh chất lượng cao của nhân vật.

25.2 Hệ thống Chiến đấu liên hoàn (Combo System)

25.2.1 Máy trạng thái đòn đánh

Nhân vật vận hành dựa trên biến punchCount để theo dõi tiến trình combo.

- **Đòn đánh thường (1-2):** Sử dụng bộ ảnh pRight, gây sát thương cơ bản.
- **Đòn Knockout (Đòn thứ 3):** Chuyển sang bộ ảnh kRight, tăng sát thương và kích hoạt cơ chế hồi phục.

25.2.2 Logic Sát thương và Phản xạ

Một điểm biến tấu đặc biệt trong mã nguồn là khả năng "phản kháng". Khi máu của BonkChoy thấp hơn mức tối đa (đang bị Zombie ăn), sát thương đầu ra sẽ được gia tăng, mô phỏng trạng thái kích động chiến đấu:

$$Dmg = \begin{cases} 15 \text{ (KO) / } 10 \text{ (Normal)} & \text{nếu } HP < MaxHP \\ 6 \text{ (KO) / } 5 \text{ (Normal)} & \text{nếu } HP = MaxHP \end{cases} \tag{26}$$

25.3 Cơ chế Hồi phục và Duy trì

25.3.1 Thuật toán Hồi máu (Lifesteal)

Để bù đắp cho việc phải tiếp xúc trực tiếp với kẻ thù, BonkChoy được trang bị khả năng hồi phục cực mạnh tại đòn đánh thứ ba:

```
if (ko) {
    punchCount = 0; // Reset combo
    this.hp += (int)(maxHp * 0.35); // Hoi 35% luong mau toi da
    if (this.hp > maxHp) this.hp = maxHp;
}
```

Cơ chế này tạo ra một vòng lặp sinh tồn: Tấn công càng nhanh, khả năng sống sót trước đám đông Zombie càng cao.

25.4 Kết Luận Sơ Bộ

Lớp `BonkChoy` minh chứng cho khả năng thiết lập logic AI đối kháng chi tiết. Việc kết hợp giữa hoạt ảnh, âm thanh (*bonk.mp3*) và các chỉ số thay đổi linh hoạt theo trạng thái mang lại chiều sâu cho nhân vật, biến nó thành một đơn vị phòng thủ chiến lược trong *PvZ: Thìa Vàng*.

26 BonkChoy2

BonkChoy2 là biến thể nâng cấp (Tier 2) của đơn vị cận chiến BonkChoy. Vẫn giữ nguyên các đặc tính kỹ thuật cốt lõi từ phiên bản gốc, nhưng lớp này được tối ưu hóa mạnh mẽ về chỉ số vật lý và tần suất tấn công. Các cải tiến đáng chú ý bao gồm:

1. Gia tăng lượng máu tối đa (maxHp) lên mức 1000 đơn vị.
2. Tốc độ khung hình hoạt ảnh tấn công được đẩy nhanh gấp đôi (delay = 20ms).
3. Giảm thời gian chờ giữa các đợt gây sát thương (delay = 200ms).
4. Duy trì cơ chế hồi phục đột biến (35%) tại đòn đánh thứ ba.

26.1 Sự Khác Biệt về Hiệu Năng Chiến Đấu

26.1.1 Tối ưu hóa Tốc độ Tấn công (Attack Speed)

Trong phiên bản BonkChoy2, tốc độ tấn công được tinh chỉnh để tạo ra áp lực lớn lên kẻ thù. So với phiên bản tiền nhiệm, thời gian thực thi một chu kỳ đòn đánh đã được rút ngắn đáng kể:

$$T_{attack} = \text{delay}_{anim} \times \text{length}(pRight) \approx 20ms \times 10 = 200ms \tag{27}$$

Việc rút ngắn thời gian xử lý sát thương giúp nhân vật kích hoạt trạng thái *Knockout* (KO) nhanh hơn, từ đó tăng tần suất hồi phục máu trong giao tranh.

26.2 Logic Combo và Hồi Phục Nâng Cao

26.2.1 Cơ chế Sát thương Phản xạ (Berserker Logic)

Tương tự phiên bản trước, BonkChoy2 sử dụng logic sát thương tăng cường dựa trên trạng thái bị tấn công (beingEaten). Tuy nhiên, với lượng máu 1000, đơn vị này có thể trụ vững lâu hơn trong trạng thái cường hóa sát thương.

- **Trạng thái KO (Sát thương kết liễu):** Gây 15 sát thương nếu đang bị ăn, 6 sát thương nếu ổn định.
- **Tần suất hồi máu:** Tăng cao do tốc độ tích lũy punchCount nhanh hơn.

26.2.2 Thuật toán Hồi phục Tuyệt đối

Giá trị hồi phục của BonkChoy2 đạt hiệu quả cực cao nhờ vào tổng lượng máu lớn:

$$HP_{gain} = 1000 \times 0.35 = 350 \text{ đơn vị máu mỗi combo} \tag{28}$$

Điều này biến BonkChoy2 thành một "bức tường sống"có khả năng tự duy trì sự tồn tại ngay cả khi đối đầu với các đợt Zombie đông đảo (*Zombie Hordes*).

26.3 Xử Lý Hoạt Ảnh Đa Trạng Thái

26.3.1 Đồng bộ hóa Hoạt ảnh và Logic Sát thương

Hàm playLoop được cấu hình với độ trễ thấp (20ms cho tấn công và 40ms cho nghỉ) đảm bảo rằng chuyển động của nhân vật trông rất nhanh nhẹn nhưng vẫn giữ được độ mượt mà cần thiết. Việc phối hợp giữa hoạt ảnh pRight (đấm thường) và kRight (đấm móc) được kiểm soát chặt chẽ bởi biến cờ isKO.

26.4 Kết Luận Sơ Bộ

`BonkChoy2` là minh chứng cho việc tái sử dụng mã nguồn hiệu quả trong lập trình hướng đối tượng. Bằng cách thay đổi các hằng số về thời gian và chỉ số máu, dự án đã tạo ra một thực thể có chiều sâu gameplay hoàn toàn khác biệt, đóng vai trò là quân bài chủ lực trong các chiến thuật phòng thủ cận chiến của *PvZ: Thìa Vàng*.

27 PotatoMine

PotatoMine là một đơn vị phòng thủ đặc biệt thuộc nhóm bẫy nổ, kế thừa từ lớp Plant. Khác với các loại cây tấn công trực tiếp, PotatoMine yêu cầu một khoảng thời gian chuẩn bị để chuyển sang trạng thái sẵn sàng chiến đấu. Các đặc tính kỹ thuật cốt lõi bao gồm:

1. Cơ chế kích ngòi (*Arming mechanism*) dựa trên thời gian thực.
2. Hệ thống hoạt ảnh hai giai đoạn: Chờ (*Underground*) và Sẵn sàng (*Armed*).
3. Logic nổ vùng (*Explosion*) kích hoạt qua va chạm vật lý với Zombie.
4. Khả năng miễn nhiễm sát thương tạm thời trong trạng thái đã kích ngòi.

27.1 Cơ Chế Kích Ngòi (Arming Logic)

27.1.1 Quản lý thời gian chuẩn bị

Để tái hiện độ trễ đặc trưng, lớp sử dụng biến deltaTime2 để đo lường khoảng thời gian kể từ khi thực thể được đặt xuống lưới. Chỉ khi thời gian chờ vượt ngưỡng 10 giây ($10.000\,ms$), bẫy mới chuyển sang trạng thái hoạt động.

$$\text{Armed} = \text{true} \iff (T_{current} - T_{initial}) \geq 10.000\,ms \tag{29}$$

27.1.2 Chuyển đổi trạng thái hiển thị

Ngay khi biến armed chuyển sang true, một chuỗi các sự kiện logic và hình ảnh được kích hoạt:

- **Âm thanh:** Phát file hiệu ứng *"dirt_rise.mp3"* một lần duy nhất.
- **Hoạt ảnh:** Chuyển từ bộ ảnh ẩn dưới đất sang bộ ảnh arm (trồi lên mặt đất). Sau khi hoạt ảnh arm hoàn tất (frame > 2), thực thể duy trì hoạt ảnh idle của trạng thái sẵn sàng.

27.2 Logic Va Chạm và Kích Nổ

27.2.1 Kiểm tra vùng nổ (Hitbox Detection)

Tương tự các đơn vị tấn công khác, PotatoMine truy vấn danh sách Zombie trên cùng hàng thông qua zombieRow. Tuy nhiên, thay vì bắn đạn, nó tự kiểm tra khoảng cách tương đối giữa chính nó và các mục tiêu di động.

Listing 16: Thuật toán kiểm tra va chạm kích nổ

```
for (Zombie i : PlayScene.level.zombieRow.get(getYPos())) {
    if (Math.abs(i.getX() - getX()) < 28) {
        // Khoi tao thuc the no tai vi tri va cham
        getWorld().addObject(new Explosion(...), getX(), getY()
            -25);
        // Tu huy khoi he thong luoi
        ((PlayScene)getWorld()).GridManager.removePlant(getXPos(),
            getYPos());
        return;
    }
}
```

27.2.2 Phân tách logic: Lớp Explosion

Một điểm thiết kế đáng chú ý là việc `PotatoMine` không trực tiếp trừ máu Zombie. Thay vào đó, nó tạo ra một thực thể `Explosion` mới. Cách tiếp cận này giúp tách biệt logic hiển thị hiệu ứng nổ và logic tính toán sát thương diện rộng, cho phép dễ dàng tái sử dụng hiệu ứng nổ cho các loại cây khác (như Cherry Bomb).

27.3 Tương Tác Đặc Biệt

27.3.1 Cơ chế miễn nhiễm sát thương

Trong phương thức `hit()`, `PotatoMine` có một điều kiện kiểm tra trạng thái:

- **Chưa kích ngòi (!armed):** Nhận sát thương bình thường từ Zombie.
- **Đã kích ngòi (armed):** Không nhận sát thương. Điều này đảm bảo bẫy nổ chắc chắn sẽ kích hoạt ngay khi Zombie chạm vào mà không bị tiêu diệt bởi các đòn tấn công trước đó.

27.4 Kết Luận Sơ Bộ

Lớp `PotatoMine` thể hiện sự phối hợp nhịp nhàng giữa quản lý thời gian thực và xử lý sự kiện dựa trên trạng thái (*State-based programming*). Việc sử dụng biến cờ (`armed`, `playOnce`) giúp kiểm soát luồng hoạt ảnh phức tạp một cách chính xác, mang lại trải nghiệm gameplay trung thành với phiên bản gốc.

28 Zombie

Lớp Zombie đóng vai trò là lớp cơ sở cho toàn bộ hệ thống kẻ thù trong dự án *PvZ: Thìa Vàng*. Được xây dựng dựa trên lớp SpriteAnimator, Zombie không chỉ đảm nhận việc diễn họa các hành động di chuyển, tấn công mà còn quản lý một hệ thống trạng thái phức tạp nhằm mô phỏng các giai đoạn tổn thương và tử vong một cách chân thực.

28.1 Máy trạng thái và Logic hành vi

Thực thể Zombie vận hành thông qua một máy trạng thái (State Machine) điều khiển bởi các biến logic, cho phép chuyển đổi mượt mà giữa các hành vi:

- **Trạng thái Di chuyển (Walking):** Trạng thái mặc định khi thực thể tiến về phía bên trái màn hình dựa trên biến walkSpeed.
- **Trạng thái Tấn công (Eating):** Kích hoạt khi phát hiện có Plant trong phạm vi va chạm. Thực thể sẽ dừng di chuyển và thực hiện trừ máu mục tiêu theo chu kỳ khung hình.
- **Trạng thái Trọng thương (Headless):** Khi lượng máu giảm xuống ngưỡng tử vong, thực thể sẽ thực hiện rơi các bộ phận vật lý (Head) nhưng vẫn có thể tiếp tục hành động trong một khoảng thời gian ngắn.
- **Trạng thái Tử vong (Final Death):** Thực thể hoàn toàn ngừng hoạt động, được thay thế bằng hiệu ứng ngã (fallingZombie) và giải phóng khỏi bộ nhớ.

28.2 Cơ chế Tương tác Hệ thống

28.2.1 Hệ thống Nhận diện Mục tiêu theo Hàng

Để tối ưu hóa hiệu suất, Zombie truy vấn trực tiếp vào mảng Board của GridManager. Thay vì sử dụng các hàm kiểm tra va chạm liên tục của Engine, thực thể chỉ thực hiện quét các ô lưới trong hàng (yIdx) tương ứng.

Listing 17: Logic nhận diện mục tiêu dựa trên khoảng cách Euclide

```
if (p != null && p.getWorld() != null) {
    if (Math.abs(p.getX() - getX() + 5) < 35) {
        eating = true;
        target = p;
        return true;
    }
}
```

28.2.2 Logic Tấn công Chu kỳ (Frame-based Damage)

Để đồng bộ hóa âm thanh và sát thương với hành động nhai trên Sprite, sát thương chỉ được áp dụng tại các khung hình cụ thể trong chuỗi hoạt ảnh (frame == 5 hoặc frame == 2). Điều này đảm bảo tính nhất quán giữa thị giác và cơ chế gameplay.

28.3 Xử lý Tử vong đa giai đoạn

Một trong những điểm nổi bật của dự án là việc chia quá trình tử vong thành nhiều bước (*Multi-stage Death*):

1. **Khởi phát:** Khi `hp <= 0`, `isAlive` chuyển sang `false`, thực thể bị loại bỏ khỏi danh sách quản lý hàng (`removeFromRow`) để đạn đạo không còn nhắm vào.
2. **Phân rã:** Thực thể sinh ra một đối tượng `Head` (vật lý rơi) và chuyển sang bộ hoạt ảnh `headless`.
3. **Kết thúc:** Sau khi chuỗi hoạt ảnh kết thúc, một thực thể trung gian `fallingZombie` được tạo ra để thực hiện hiệu ứng ngã xuống đất, đồng thời thực thể `Zombie` gốc chính thức bị xóa khỏi `World`.

28.4 Kết luận kỹ thuật

Lớp `Zombie` thể hiện khả năng điều phối phức tạp giữa logic hướng đối tượng và xử lý đồ họa. Việc áp dụng cơ chế nhận diện mục tiêu thông qua `GridManager` giúp trò chơi duy trì tốc độ khung hình ổn định ngay cả khi số lượng kẻ thù xuất hiện lớn, đồng thời tạo ra một hệ thống đối kháng có chiều sâu và tính tương tác cao.

29 BasicZombie

`BasicZombie` là đơn vị kẻ thù tiêu chuẩn trong *PvZ: Thìa Vàng*, kế thừa trực tiếp từ lớp `Zombie`. Mặc dù là thực thể cơ bản nhất, lớp này được thiết lập để minh chứng cho cơ chế "Tổn thương theo ngưỡng máu"(Threshold-based Damage), nơi hình thái và hành vi của thực thể thay đổi dựa trên lượng `hp` còn lại.

29.1 Cơ chế Biến đổi Hình thái (Morphology Logic)

29.1.1 Hệ thống ngưỡng máu (HP Thresholds)

Thực thể vận hành dựa trên một điều kiện kiểm tra ngưỡng máu quan trọng tại mức $50\,HP$. Sự phân hóa này chia vòng đời của `BasicZombie` thành hai giai đoạn rõ rệt:

1. **Giai đoạn Nguyên vẹn ($HP > 50$):** Sử dụng bộ hoạt ảnh đầy đủ (`walk`, `eat`). Tốc độ di chuyển được thiết lập ngẫu nhiên trong khoảng $[22, 28]$ đơn vị để tạo ra sự không đồng nhất trong hàng ngũ kẻ thù.
2. **Giai đoạn Tổn thương ($HP \leq 50$):**
   - Kích hoạt sự kiện vật lý đơn lẻ: Sinh ra đối tượng `Arm` (cánh tay rơi) thông qua lớp `PhysicalEntities`.
   - Chuyển đổi bộ ảnh sang các trạng thái mất chi (`armless`, `armlesseat`).
   - Phát hiệu ứng âm thanh đặc trưng (`limbs_pop.mp3`).

$$\text{State} = \begin{cases} \text{Intact} & \text{nếu } HP > 50 \\ \text{Armless} & \text{nếu } HP \leq 50 \end{cases} \tag{30}$$

29.2 Đa hình và Ghi đè phương thức (Method Overriding)

29.2.1 Tùy biến hành vi Cập nhật (Update)

Bằng cách ghi đè phương thức `update()`, `BasicZombie` thực hiện điều phối các bộ hoạt ảnh dựa trên hai biến trạng thái: `isEating()` và `fallen`. Việc sử dụng hàm `playEating()` từ lớp cha đảm bảo sát thương được áp dụng đồng bộ với nhịp nhai của hoạt ảnh.

29.2.2 Cơ chế Phản hồi Va chạm (Hit Feedback)

Phương thức `hit()` được mở rộng để tích hợp hệ thống âm thanh ngẫu nhiên (`splat.mp3`) và hiệu ứng `hitFlash`. Điểm phức tạp ở đây là việc lựa chọn tài nguyên hình ảnh để nháy sáng (*Flash*) phải khớp hoàn toàn với trạng thái hình thái hiện tại của thực thể:

Listing 18: Logic lựa chọn Sprite cho hiệu ứng Flash

```
if (!fallen) {
    // Neu con nguyen ven, nhay sang anh walking hoac eating
    hitFlash(eating ? eat : walk, eating ? "zombieeating" : "
        zombiewalk");
} else {
    // Neu da mat tay, nhay sang bo anh armless tuong ung
    hitFlash(eating ? armlesseat : armless, eating ? "
        armlesszombieeating" : "armlesszombie");
```

```
}
```

29.3 Thông số Kỹ thuật và Cân bằng

Bảng 6: Chỉ số cơ bản của BasicZombie

Thông số	**Giá trị / Mô tả**
Máu tối đa (`maxHp`)	150 đơn vị
Sát thương (`damage`)	5 đơn vị mỗi nhịp
Tốc độ di chuyển	Random(22, 28)
Độ trễ hoạt ảnh	350ms (Di chuyển) / 200ms (Tấn công)

29.4 Kết luận kỹ thuật

`BasicZombie` là minh chứng cho sức mạnh của tính kế thừa trong dự án. Chỉ với việc thay đổi các hằng số và ghi đè logic cập nhật, nhóm đã tạo ra một thực thể có khả năng phản hồi trực quan cao với các tác động từ người chơi, đồng thời duy trì cấu trúc mã nguồn gọn gàng và dễ mở rộng cho các loại Zombie phức tạp hơn trong tương lai.

30 Brickhead

`Brickhead` là một biến thể kẻ thù cấp cao, đại diện cho cơ chế phòng ngự đa tầng trong dự án *PvZ: Thìa Vàng*. Với lượng máu lên tới 300 đơn vị (gấp đôi `BasicZombie`), thực thể này yêu cầu một hệ thống quản lý hoạt ảnh chi tiết để phản hồi trực quan quá trình hư hại của lớp giáp bảo vệ (viên gạch) trước khi lộ ra hình thái nguyên bản của kẻ thù.

30.1 Cơ chế Giáp đa tầng và Phân rã vật lý

30.1.1 Hệ thống ngưỡng hư hại (Degradation Thresholds)

Thực thể `Brickhead` vận hành dựa trên một cấu trúc điều kiện phân tầng phức tạp, điều phối sự thay đổi của các bộ Sprite dựa trên các mốc `hp` cụ thể:

1. **Giai đoạn Giáp nguyên vẹn ($HP > 200$):** Sử dụng bộ ảnh `brickheadwalk` và `brickheadeat`.
2. **Giai đoạn Giáp nứt ($200 \geq HP > 100$):** Chuyển sang bộ ảnh `brickheadwalkd` (Degraded).
3. **Giai đoạn Giáp vỡ nặng ($100 \geq HP > 50$):** Chuyển sang bộ ảnh `brickheadwalkdd` (Double Degraded).
4. **Giai đoạn Phá hủy giáp ($HP = 50$):** Kích hoạt sự kiện vật lý sinh ra đối tượng `Brick` (vật lý rơi) và chuyển về hình thái Zombie tiêu chuẩn.

$$\text{Visual State} = \begin{cases} \text{Brickhead (Full)} & HP \in (200, 300] \\ \text{Brickhead (Damaged)} & HP \in (100, 200] \\ \text{Brickhead (Broken)} & HP \in (50, 100] \\ \text{Standard Zombie} & HP \leq 50 \end{cases} \tag{31}$$

30.2 Thiết kế Code và Tối ưu hóa Hành vi

30.2.1 Đóng gói logic hoạt ảnh (handleAnimation)

Nhóm đã triển khai phương thức hỗ trợ `handleAnimation()` để đóng gói các logic lặp lại giữa việc di chuyển và tấn công. Điều này không chỉ giúp mã nguồn ngắn gọn hơn mà còn đảm bảo tính nhất quán trong việc cập nhật tọa độ và xử lý sát thương mục tiêu thông qua `playEating()`.

30.2.2 Phản hồi tương tác đa điểm (Hit Feedback)

Phương thức `hit()` của `Brickhead` là một minh chứng cho sự tỉ mỉ trong việc xây dựng trải nghiệm người dùng (UX). Hiệu ứng `hitFlash` được cấu hình để nháy sáng đúng tập tin hình ảnh tương ứng với mức độ hư hại hiện tại của giáp:

Listing 19: Logic truy vấn Sprite cho hiệu ứng nháy sáng theo trạng thái giáp

```
if (hp > 200) {
    hitFlash(eating ? brickheadeat : brickheadwalk, ...);
} else if (hp > 100) {
    hitFlash(eating ? brickheadeatd : brickheadwalkd, ...);
```

```
}
// He thong tiep tuc phan cap den khi giap vo hoan toan
```

30.3 Thông số Kỹ thuật và Cân bằng Game

Bảng 7: Bảng chỉ số chiến đấu của Brickhead

Thông số	**Giá trị / Mô tả**
Máu tối đa (`maxHp`)	300 đơn vị (Cấp độ: Heavy Tanker)
Sát thương (`damage`)	150 đơn vị (Sát thương cực lớn lên hàng phòng thủ)
Tốc độ di chuyển	Random(22, 28)
Vật lý rơi đi kèm	`Brick`, `Arm`, `Head`

30.4 Kết luận kỹ thuật

Lớp `Brickhead` thể hiện sự làm chủ kỹ thuật trong việc quản lý tài nguyên đồ họa lớn và logic trạng thái chồng chéo. Việc kết hợp giữa âm thanh (`splat.mp3`, `limbs_pop.mp3`) và các hiệu ứng vật lý rơi khiến thực thể này trở thành một trong những điểm nhấn về mặt thị giác và thử thách chiến thuật quan trọng nhất trong hệ thống kẻ thù của dự án.

31 Buckethead

`Buckethead` là một biến thể kẻ thù thuộc lớp `Zombie` có khả năng chống chịu cao (Tanker). Đặc điểm nhận dạng chính là chiếc xô sắt trên đầu, đóng vai trò như một lớp giáp vật lý. Thực thể này có lượng máu cơ bản là 450 đơn vị, yêu cầu hệ thống quản lý trạng thái phức tạp để mô tả quá trình hư hại theo thời gian thực.

31.1 Cơ chế Giáp và Phân rã vật lý

31.2 Hệ thống ngưỡng hư hại (Degradation Thresholds)

Sự thay đổi về ngoại hình của `Buckethead` được điều phối dựa trên các ngưỡng `hp` (Health Points) cụ thể. Nhóm đã triển khai logic phân tầng để thay đổi bộ Sprite tương ứng:

- **Xô nguyên vẹn ($HP > 375$):** Sử dụng bộ Sprite gốc.
- **Xô móp nhẹ ($375 \geq HP > 200$):** Chuyển sang trạng thái `bucketheadwalkd`.
- **Xô hỏng nặng ($200 \geq HP > 100$):** Chuyển sang trạng thái `bucketheadwalkdd`.
- **Mất giáp ($HP \leq 100$):** Sinh ra đối tượng vật lý `Bucket` và trở thành Zombie thường.
- **Rụng chi ($HP \leq 50$):** Sinh ra đối tượng `Arm` và chuyển sang trạng thái `armless`.

$$\text{State}(hp) = \begin{cases} \text{Full Armor} & hp \in (375, 450] \\ \text{Damaged} & hp \in (200, 375] \\ \text{Broken} & hp \in (100, 200] \\ \text{Standard} & hp \in (50, 100] \\ \text{Armless} & hp \leq 50 \end{cases} \tag{32}$$

31.3 Thiết kế Logic điều khiển

31.3.1 Tối ưu hóa hoạt ảnh (handleAnimation)

Để tránh lặp code, phương thức `handleAnimation()` được thiết kế để xử lý chung cho cả trạng thái di chuyển và tấn công. Logic này đảm bảo tốc độ khung hình và âm thanh được đồng bộ hóa dựa trên phương thức `isEating()`.

31.3.2 Phản hồi tương tác (Hit Feedback)

Hệ thống âm thanh được phân tách dựa trên biến `boolean bucket`. Khi lớp giáp còn tồn tại, âm thanh va chạm kim loại (*shieldhit.mp3*) sẽ được ưu tiên. Sau khi mất giáp, hệ thống tự động chuyển sang âm thanh va chạm thực thể (*splat.mp3*).

31.4 Thông số Kỹ thuật

31.5 Kết luận

Lớp `Buckethead` thể hiện sự làm chủ kỹ thuật trong việc quản lý tài nguyên đồ họa và logic trạng thái. Việc kết hợp giữa âm thanh, hình ảnh nháy sáng (`hitFlash`) và các mảnh vỡ vật lý tạo nên trải nghiệm game sống động và trực quan cho người chơi.

Thuộc tính	Thông số / Mô tả
Máu tối đa (maxHp)	450 HP
Sát thương (damage)	35 DPS
Tốc độ di chuyển	Biến thiên ngẫu nhiên trong khoảng $[22, 28]$
Vật lý rơi (Props)	Bucket (Xô), Arm (Tay), Head (Đầu)
Âm thanh	Phân cấp theo vật liệu (Kim loại → Thực thể)

Bảng 8: Chỉ số chiến đấu của Buckethead Zombie

32 Conehead

Conehead là một biến thể nâng cấp từ lớp Zombie tiêu chuẩn, được trang bị một chiếc nón giao thông đóng vai trò lớp giáp bảo vệ bổ sung. Với lượng máu tối đa đạt 400 đơn vị, thực thể này đóng vai trò là đơn vị chống chịu trung cấp, giúp tạo ra áp lực lên hệ thống phòng thủ của người chơi trong giai đoạn đầu và giữa trận đấu.

32.1 Cơ chế Giáp và Phân rã vật lý

32.1.1 Hệ thống quản lý trạng thái theo ngưỡng máu

Thực thể Conehead sử dụng một hệ thống điều kiện phân tầng để thay đổi bộ nhận diện đồ họa (Sprite) dựa trên mức độ hư hại của chiếc nón. Nhóm đã triển khai logic cập nhật trạng thái tại phương thức update() như sau:

- **Giai đoạn Giáp nguyên vẹn ($HP > 232$):** Sử dụng bộ Sprite coneheadwalk/eat.
- **Giai đoạn Giáp móp nhẹ ($232 \geq HP > 166$):** Chuyển sang trạng thái đồ họa coneheadwalkd.
- **Giai đoạn Giáp hư hại nặng ($166 \geq HP > 100$):** Chuyển sang trạng thái đồ họa coneheadwalkdd.
- **Phá hủy giáp ($HP \leq 100$):** Biến cone chuyển về false, sinh ra đối tượng vật lý Cone tại tọa độ tương ứng và trở về hình thái Zombie cơ bản.
- **Trạng thái rụng chi ($HP \leq 50$):** Kích hoạt biến fallen, sinh ra đối tượng Arm và chuyển sang bộ ảnh mất cánh tay (armless).

$$\text{Visual State}(hp) = \begin{cases} \text{Cone (New)} & hp \in (232, 400] \\ \text{Cone (Damaged)} & hp \in (166, 232] \\ \text{Cone (Broken)} & hp \in (100, 166] \\ \text{Standard} & hp \in (50, 100] \\ \text{Armless} & hp \leq 50 \end{cases} \tag{33}$$

32.2 Thiết kế Logic điều khiển

32.2.1 Trừu tượng hóa hoạt ảnh (handleAnimation)

Phương thức `handleAnimation()` được thiết kế để xử lý đa luồng hành vi. Logic tự động kiểm tra trạng thái thông qua `isEating()`:

- Nếu đang di chuyển: Thực hiện phương thức `move()` với vận tốc `walkSpeed` và hoạt ảnh di chuyển 350ms.
- Nếu đang tấn công: Chuyển sang hoạt ảnh ăn 200ms và kích hoạt chuỗi âm thanh `playEating()`.

32.2.2 Phản hồi tương tác vật lý (Hit Feedback)

Khi thực thể nhận sát thương thông qua phương thức `hit()`, hệ thống sẽ thực hiện đồng thời hai phản hồi:

1. **Âm thanh:** Nếu lớp giáp còn tồn tại (`cone == true`), âm thanh nhựa (*plastichit.mp3*) sẽ phát cùng lúc với âm thanh va chạm thực thể (*splat.mp3*).
2. **Thị giác:** Phương thức `hitFlash()` được gọi với tham số hình ảnh khớp chính xác với trạng thái máu hiện tại, đảm bảo tính nhất quán về mặt thị giác ngay cả khi Zombie đã mất đầu (`headless`).

32.3 Thông số Kỹ thuật

Thông số	**Giá trị / Mô tả**
Máu tối đa (`maxHp`)	400 HP
Sát thương (`damage`)	30 DPS
Tốc độ di chuyển	Random.Double(22, 28)
Đối tượng vật lý rơi	`Cone`, `Arm`, `Head`
Hiệu ứng âm thanh	`plastichit.mp3`, `limbs_pop.mp3`, `splat.mp3`

Bảng 9: Chỉ số chiến đấu của Conehead Zombie

32.4 Kết luận

Thực thể `Conehead` là minh chứng cho việc sử dụng hiệu quả cấu trúc dữ liệu mảng (`GreenfootImage[]`) và logic điều kiện phân tầng. Việc kết hợp phản hồi âm thanh nhựa đặc trưng giúp người chơi dễ dàng nhận biết loại kẻ thù đang đối mặt, đồng thời tạo ra chiều sâu về mặt tương tác vật lý trong môi trường trò chơi.

33 Projectile

Lớp `Projectile` quản lý toàn bộ các thực thể đạn đạo trong trò chơi, kế thừa từ `SpriteAnimator`. Lớp này đảm nhận vai trò cầu nối tương tác giữa hệ thống phòng thủ (`Plant`) và hệ thống kẻ thù (`Zombie`), với các chức năng chính:

1. Điều khiển chuyển động tịnh tiến với vận tốc tùy biến (`speed`).
2. Xử lý va chạm dựa trên không gian hàng (*Row-based Collision*).
3. Quản lý hiệu ứng hoạt ảnh khi va chạm (*Hit animation*).
4. Tối ưu hóa bộ nhớ bằng cách tự động giải phóng thực thể khi ra khỏi biên hoặc kết thúc hoạt ảnh.

33.1 Cơ Chế Va Chạm và Tối Ưu Hiệu Năng

33.1.1 Hệ thống Va chạm theo hàng (Row-based)

Thay vì kiểm tra va chạm với toàn bộ các đối tượng `Zombie` trên bản đồ (gây tốn tài nguyên $O(n)$), `Projectile` tận dụng cấu trúc dữ liệu `zombieRow` của `Level`. Đạn chỉ thực hiện kiểm tra các thực thể nằm trong danh sách của hàng tương ứng (`yPos`).

Listing 20: Thuật toán kiểm tra va chạm tối ưu

```
List<Zombie> row = world.level.zombieRow.get(yPos);
for (Zombie z : new ArrayList<>(row)) {
    if (Math.abs(z.getX() - getX()) < 30) {
        z.hit(damage);
        this.hit = true;
        break;
    }
}
```

Việc sử dụng `Math.abs(z.getX() - getX()) < 30` cho phép xác định va chạm dựa trên khoảng cách thay vì dựa vào các hàm `getOneIntersectingObject` mặc định của Greenfoot, giúp việc phản hồi va chạm chính xác và mượt mà hơn.

33.2 Xử Lý Hoạt Ảnh Va Chạm

33.2.1 Máy trạng thái đơn giản

Thực thể `Projectile` vận hành dựa trên hai trạng thái chính:

- **Trạng thái Bay:** Di chuyển liên tục theo trục X và kiểm tra va chạm.
- **Trạng thái Va chạm (`hit = true`):** Ngừng di chuyển, chuyển sang chuỗi hoạt ảnh "vỡ"(ví dụ: hạt đậu vỡ ra). Thực thể chỉ bị xóa sau khi hoạt ảnh này hoàn tất (`frame >= frameCount - 1`).

Cơ chế này mang lại trải nghiệm thị giác tốt hơn, giúp người chơi nhận biết rõ ràng việc đạn đã tác động thành công lên kẻ thù.

33.3 Quản Lý Tài Nguyên

33.3.1 Tự động giải phóng (Auto-removal)

Để duy trì hiệu suất ổn định cho trò chơi, đặc biệt là trong các đợt tấn công lớn với hàng trăm viên đạn, lớp thực hiện kiểm tra biên (*Edge detection*):

```
if (isAtEdge()) {
    world.removeObject(this);
    return;
}
```

Điều này đảm bảo các viên đạn bay trượt mục tiêu sẽ không tồn tại vĩnh viễn trong bộ nhớ, tránh tình trạng giật lag (*memory leak*) sau thời gian dài chơi game.

33.4 Kết Luận Sơ Bộ

Lớp `Projectile` là một ví dụ điển hình về việc kết hợp giữa logic vật lý đơn giản và tối ưu hóa cấu trúc dữ liệu. Việc sử dụng kế thừa từ `SpriteAnimator` cho phép lớp này vừa có thể di chuyển chính xác, vừa có thể thực hiện các hiệu ứng hình ảnh phức tạp mà không làm rối mã nguồn xử lý logic.

34 ProjectileVariants

34.1 Tóm Tắt

Dựa trên lớp cơ sở `Projectile`, dự án triển khai nhiều loại đạn đạo với các đặc tính vật lý và cơ chế va chạm khác nhau. Sự phân hóa này không chỉ dừng lại ở chỉ số sát thương mà còn mở rộng sang quỹ đạo bay và khả năng xuyên thấu (*Piercing*), tạo nên chiều sâu chiến thuật cho hệ thống phòng thủ.

34.2 Phân loại và Chỉ số Kỹ thuật

Các loại đạn cơ bản kế thừa trực tiếp logic va chạm của lớp cha, tập trung vào việc tùy biến sát thương và tốc độ bay.

Bảng 10: Thông số kỹ thuật các loại đạn đạo cơ bản

Loại đạn	Sát thương	Vận tốc	Hoạt ảnh (Frames)	Đặc tính
`Pea`	15	6	3	Đạn đậu tiêu chuẩn
`FirePea`	40	10	3	Sát thương cao, tốc độ nhanh
`Needle`	8	3	1	Tốc độ chậm, bắn tỉa

34.3 Cơ chế Đạn xuyên thấu và Quỹ đạo phức tạp

Các lớp `Needle2` và `Needle3` đại diện cho bước tiến về logic lập trình, ghi đè hàm `act()` để triển khai các hành vi đặc biệt.

34.3.1 Quỹ đạo bay theo góc (Offset Trajectory)

Thay vì chỉ di chuyển theo trục X, các biến thể `Needle` có khả năng di chuyển theo trục Y để đạt tới một mục tiêu (`targetY`) định trước. Điều này mô phỏng các kỹ năng bắn lan hoặc bắn chéo.

$$y_{next} = y_{current} + \text{verticalStep} \quad \text{với verticalStep} \in \{-2, 2\} \tag{34}$$

34.3.2 Thuật toán Xuyên thấu với HashSet

Khác với đạn thông thường biến mất ngay khi chạm mục tiêu đầu tiên, đạn xuyên thấu cần một cơ chế để tránh việc gây sát thương nhiều lần lên cùng một đối tượng trong các frame liên tiếp. Nhóm đã sử dụng cấu trúc dữ liệu `HashSet` để lưu trữ các mục tiêu đã trúng đòn.

Listing 21: Logic xuyên thấu sử dụng HashSet

```java
private Set<Zombie> hitZombies = new HashSet<>();

private void checkPiercingCollision() {
    List<Zombie> zombies = getIntersectingObjects(Zombie.class);
    for (Zombie z : zombies) {
        if (!hitZombies.contains(z)) {
            z.hit(pierceDamage);
```

```
                hitZombies.add(z); // Danh dau muc tieu da bi trung
                    dan
            }
        }
}
```

Lợi ích: Đảm bảo mỗi Zombie chỉ nhận sát thương đúng một lần duy nhất từ một viên đạn, ngay cả khi viên đạn đó đang nằm đè lên hình ảnh của Zombie trong nhiều frame.

34.4 So sánh Hiệu quả Chiến thuật

Việc thiết kế đa dạng các loại đạn cho phép xây dựng các tổ hợp phòng thủ tối ưu:

- `FirePea`**:** Phù hợp để tiêu diệt nhanh các mục tiêu đơn lẻ có lượng máu lớn.
- `Needle2/3`**:** Hiệu quả cao khi đối đầu với đám đông (*Horde*) nhờ khả năng xuyên qua nhiều kẻ thù và quỹ đạo bay bao phủ phạm vi rộng trên lưới lục giác.

34.5 Kết Luận Sơ Bộ

Hệ thống đạn đạo minh chứng cho khả năng mở rộng của kiến trúc hướng đối tượng trong dự án. Việc kết hợp giữa các lớp kế thừa đơn giản và các lớp có logic phức tạp (*Piercing & Trajectory*) tạo nên một hệ thống chiến đấu linh hoạt, đáp ứng tốt yêu cầu về mặt gameplay của thể loại *Auto Chess*.

35 Explosion

Lớp `Explosion` là một thực thể đặc biệt kế thừa từ `SpriteAnimator`, đóng vai trò xử lý các tác động vật lý và hình ảnh ngay khi một vụ nổ xảy ra (ví dụ: từ khoai tây nổ - Potato Mine). Khác với các loại đạn đạo di chuyển, `Explosion` tập trung vào việc gây sát thương diện rộng (*Area of Effect - AoE*) tại một thời điểm nhất định và quản lý vòng đời của hiệu ứng hoạt ảnh thông qua các frame hình.

35.1 Cơ chế Sát thương và Tương tác

Ngay khi đối tượng được thêm vào thế giới thông qua phương thức `addedToWorld`, hệ thống sẽ thực hiện quét các mục tiêu nằm trong phạm vi ảnh hưởng thay vì chờ đợi vòng lặp `act()`.

- **Phản hồi âm thanh:** Kích hoạt file âm thanh `potato_mine.mp3` với cường độ 80 để tăng tính chân thực cho hành động nổ.
- **Thuật toán quét mục tiêu:** Sử dụng vòng lặp ngược (*reverse for-loop*) trên danh sách `ArrayList<Zombie>` để đảm bảo tính ổn định của chỉ số mảng khi có đối tượng bị xóa hoặc thay đổi trạng thái trong quá trình lặp.

$$|x_{zombie} - x_{explosion}| < 44 \tag{35}$$

Logic xử lý: Bất kỳ Zombie nào có khoảng cách trục X so với tâm vụ nổ nhỏ hơn 44 đơn vị sẽ nhận ngay 150 đơn vị sát thương thông qua hàm `takeDmg(150)`.

35.2 Quản lý Hoạt ảnh và Vòng đời

`Explosion` tận dụng tối đa khả năng của lớp cha `SpriteAnimator` để tối ưu hóa việc giải phóng bộ nhớ.

- **Khởi tạo:** Các sprite được nạp từ tài nguyên `"spudow"` với tổng cộng 8 khung hình.
- **Vòng đời đối tượng:**
  - Trong các frame từ 0 đến 8: Thực hiện hoạt ảnh với độ trễ `100L`.
  - Sau khi kết thúc frame thứ 8: Đối tượng tự gọi `getWorld().removeObject(this)` để tự hủy.

35.3 Thông số Kỹ thuật Tóm lược

Bảng 11: Thông số kỹ thuật của thực thể Explosion

Thuộc tính	Giá trị	Ghi chú
`Base Damage`	150	Sát thương diện rộng cực cao
`AoE Range`	44 px	Phạm vi ảnh hưởng trục ngang
`Animation`	8 Frames	Tên tài nguyên: "spudow"
`Audio`	80 dB	`potato_mine.mp3`

35.4 Kết Luận Sơ Bộ

Lớp `Explosion` minh chứng cho cách thiết kế tối ưu trong Greenfoot: tách biệt logic gây sát thương (chỉ thực hiện 1 lần duy nhất khi khởi tạo) và logic hiển thị (thực hiện liên tục trong `act`). Điều này giúp giảm tải cho CPU trong khi vẫn đảm bảo độ chính xác của cơ chế Gameplay.

36 FallingObject

`FallingObject` là một lớp thực thể chuyên biệt dùng để mô phỏng các hiệu ứng vật lý rơi tự do trong game. Kế thừa từ `SpriteAnimator` và gián tiếp sử dụng hệ tọa độ của `PhysicsBody`, lớp này cung cấp:

1. Mô phỏng trọng lực thông qua biến gia tốc (`acceleration`).
2. Quỹ đạo chuyển động parabol kết hợp xoay thực thể.
3. Quản lý vòng đời dựa trên thời gian (`elapsedTime`) thay vì va chạm.
4. Hiệu ứng biến mất dần (*Fade-out*) khi kết thúc vòng đời.

36.1 Mô Hình Vật Lý và Quỹ Đạo

36.1.1 Động lực học chất điểm

Lớp sử dụng phương pháp tích phân Euler đơn giản để cập nhật vị trí. Vận tốc dọc (`vSpeed`) được cộng dồn bởi gia tốc sau mỗi khung hình, tạo ra cảm giác vật nặng đang rơi dưới tác động của trọng lực.
Công thức cập nhật trạng thái tại thời điểm t:

$$v_y(t) = v_y(t-1) + a \tag{36}$$
$$x(t) = x(t-1) + v_x \tag{37}$$
$$y(t) = y(t-1) + v_y(t) \tag{38}$$

Trong đó v_x (`hSpeed`) đóng vai trò là vận tốc ngang không đổi, giúp tạo ra quỹ đạo hình parabol khi vật thể văng ra (ví dụ: đầu zombie khi bị rụng).

36.1.2 Cơ chế xoay vật thể

Để tăng tính tự nhiên cho hiệu ứng văng, hàm `turn(rotate)` được gọi liên tục, mô phỏng mô-men xoắn của một vật thể khi bị tác động lực không đồng tâm.

36.2 Quản Lý Vòng Đời và Hiệu Ứng

36.2.1 Kiểm soát thời gian rơi (Fall Time)

Khác với các thực thể chiến đấu, `FallingObject` biến mất dựa trên thời gian thực. Hệ thống sử dụng `System.nanoTime()` để tính toán chính xác thời điểm cần dừng chuyển động vật lý và chuyển sang trạng thái giải phóng bộ nhớ.

Listing 22: Logic cập nhật trạng thái rơi

```
if (elapsedTime < fallTime) {
    double x = getExactX() + hSpeed;
    double y = getExactY() + vSpeed;
    setLocation(x, y);
    vSpeed += acceleration; // Tang toc do roi
} else {
    checkDeath(); // Chuyen sang hieu ung bien mat
}
```

36.2.2 Hiệu ứng Fade-out (CheckDeath)

Khi thời gian rơi kết thúc, thay vì biến mất đột ngột, thực thể thực hiện giảm dần độ trong suốt (*Transparency*).

- **Tốc độ giảm:** 3 đơn vị/frame.
- **Điều kiện giải phóng:** Khi `transparency` ≤ 0, đối tượng tự gỡ bỏ khỏi `World`.

Cơ chế này giúp giảm bớt áp lực về mặt thị giác cho người chơi khi có quá nhiều thực thể cùng xuất hiện trên màn hình trong các đợt tấn công lớn (*Huge Wave*).

36.3 Ứng Dụng Thực Tế

Trong *PvZ: Thìa Vàng*, `FallingObject` được sử dụng rộng rãi cho:

- **Head:** Đầu zombie văng ra khi hết máu.
- **FallingZombie:** Toàn bộ thân zombie ngã xuống sau khi rụng đầu.
- **Hạt giống/Phần thưởng:** Các vật phẩm rơi ra từ zombie đặc biệt.

36.4 Kết Luận Sơ Bộ

`FallingObject` là minh chứng cho việc áp dụng các nguyên lý vật lý cơ bản vào lập trình game 2D. Việc kết hợp giữa `PhysicsBody` (để có tọa độ chính xác) và các thuật toán gia tốc giúp lớp này tạo ra những hiệu ứng mượt mà, nâng cao đáng kể chất lượng đồ họa của dự án.

37 PhysicalEntities

37.1 Tóm lược nội dung

Dựa trên lớp cơ sở `FallingObject`, dự án triển khai một tập hợp các lớp con nhằm mô phỏng các mảnh vụn vật lý phát sinh trong quá trình tương tác giữa thực thể phòng thủ và kẻ thù. Việc tách biệt các thực thể này (như cánh tay, đầu, hoặc các loại mũ giáp) không chỉ giúp tối ưu hóa bộ nhớ thông qua tính kế thừa mà còn cho phép tùy biến linh hoạt các hằng số động lực học để phù hợp với khối lượng và đặc tính thị giác của từng đối tượng.

37.2 Phân tích Tham số Động lực học

Mỗi thực thể phụ trợ khi được khởi tạo sẽ truyền các tham số đặc trưng vào hàm dựng của lớp cha thông qua từ khóa `super()`. Các tham số này quyết định quỹ đạo văng và thời gian tồn tại của mảnh vụn.

Bảng 12: Bảng thông số hằng số vật lý của các thực thể phụ trợ

Thực thể	vSpeed	Accel	hSpeed	Rotation	LifeTime
`Arm` (Tay)	0	0.2	0.2	Random(-1,1)	350ms
`Head` (Đầu)	-3	0.2	0.7	Random(1,5)	700ms
`Brick` (Gạch)	-2	0.2	0.9	Random(1,5)	640ms
`Cone` (Nón)	-2	0.2	0.9	Random(1,5)	620ms
`Bucket` (Xô)	-2	0.2	0.9	Random(1,5)	620ms

37.3 Cơ chế Mô phỏng Quỹ đạo Văng

37.3.1 Lực đẩy ban đầu và Trọng lực

Các thực thể như `Head` hoặc `Brick` sử dụng giá trị `vSpeed` âm (từ -2 đến -3). Trong hệ tọa độ Greenfoot, điều này tạo ra một lực đẩy ngược lên trên theo trục Y tại thời điểm khởi tạo. Kết hợp với gia tốc trọng lực dương (`acceleration = 0.2`), thực thể sẽ di chuyển theo quỹ đạo parabol trước khi rơi xuống đáy màn hình, tạo cảm giác va chạm có lực tác động mạnh (*Impact force*).

37.3.2 Tính ngẫu nhiên và Sự đa dạng thị giác

Để tránh hiện tượng lặp lại đơn điệu, tham số xoay (`rotate`) được cấu hình thông qua hàm `Random.Int()`. Điều này đảm bảo mỗi khi một Zombie bị phá hủy giáp hoặc mất bộ phận, các mảnh vụn sẽ văng ra với các góc độ xoay khác nhau, gia tăng tính tự nhiên cho môi trường game.

37.4 Triển khai Lập trình Hướng đối tượng

Việc sử dụng kế thừa giúp mã nguồn của các lớp con cực kỳ tối giản, tập trung hoàn toàn vào việc định nghĩa "nhân dạng vật lý"của đối tượng đó thay vì phải viết lại logic di chuyển.

Listing 23: Triển khai lớp Head với logic kế thừa tối ưu

```
public class Head extends FallingObject {
    public Head() {
        // vSpeed, Accel, hSpeed, Rotate, LifeTime
        super(-3, 0.2, 0.7, Random.Int(1, 5), 700L);
    }
}
```

37.4.1 Quy trình kích hoạt (Trigger Mechanism)

Các thực thể này thường không tồn tại sẵn trên bản đồ mà được sinh ra thông qua các sự kiện trong lớp `Zombie`. Khi lượng máu (`HP`) của Zombie đạt đến các ngưỡng nhất định (ngưỡng mất giáp hoặc ngưỡng bị tiêu diệt), phương thức `addObject()` sẽ được gọi để chèn các mảnh vụn này vào thế giới tại tọa độ hiện thời của Zombie.

37.5 Kết luận kỹ thuật

Hệ thống `PhysicalEntities` là minh chứng cho việc ứng dụng hiệu quả các nguyên lý vật lý đơn giản vào lập trình trò chơi. Bằng cách tinh chỉnh các tham số hằng số, nhóm đã tạo ra được sự phân biệt rõ rệt giữa các loại mảnh vụn (ví dụ: Đầu văng cao hơn nhưng biến mất chậm hơn so với Cánh tay), góp phần quan trọng vào việc hoàn thiện trải nghiệm thị giác cho người chơi.

38 SunMechanics

38.1 Tóm Tắt

Tài nguyên "Mặt trời"là đơn vị kinh tế duy nhất trong *PvZ: Thìa Vàng*. Dự án triển khai hai thực thể riêng biệt kế thừa từ `FallingObject` để phục vụ các mục đích gameplay khác nhau:

1. `FallingSun`**:** Xuất hiện ngẫu nhiên từ phía trên màn hình, rơi xuống và dừng lại tại một vị trí cố định trên sân.
2. `Sun`**:** Được tạo ra từ các đơn vị sản xuất (như Sun-flower), có quỹ đạo văng hình parabol trước khi nằm yên.

38.2 Logic Thu thập và Di chuyển

38.2.1 Cơ chế Tương tác Đa phương thức

Cả hai lớp mặt trời đều hỗ trợ hai phương thức thu thập nhằm tối ưu trải nghiệm người dùng (*UX*):

- **Click chuột:** Sử dụng `Greenfoot.mouseClicked(this)`.
- **Va chạm vật lý:** Tương tác với thực thể `ThuyThan` (Hero) thông qua `isTouching()`. Điều này cho phép người chơi thu thập tài nguyên bằng cách di chuyển nhân vật đi qua, tương tự cơ chế trong các game nhập vai.

38.2.2 Thuật toán Di chuyển về Bộ đếm (Fly to Counter)

Khi được thu thập (`beenClicked = true`), thực thể chuyển sang trạng thái "bay". Thay vì biến mất ngay lập tức, nó di chuyển hướng về phía `SunCounter` trên thanh công cụ để tạo phản hồi thị giác về việc cộng tiền.

Listing 24: Thuật toán bay về mục tiêu

```java
private void flyToCounter() {
    // Xac dinh toa do diem den tu UI Seedbank
    int targetX = playScene.seedbank.sunCounter.getX();
    int targetY = playScene.seedbank.sunCounter.getY();

    turnTowards(targetX, targetY);
    move(25); // Van toc bay cao de tao cam giac hut
}
```

38.3 Quản lý Vòng đời và Trạng thái

38.3.1 Máy trạng thái của FallingSun

`FallingSun` quản lý ba trạng thái logic nghiêm ngặt:

Bảng 13: Các trạng thái của FallingSun

Trạng thái	Hành vi
`Falling`	Rơi theo chiều dọc với vận tốc cố định (2.5 px/frame).
`Landed`	Dừng lại khi đạt `fallTime`, bắt đầu đếm ngược thời gian chờ.
`Collected`	Vô hiệu hóa trọng lực, thực hiện hành trình bay về UI.

38.3.2 Thời gian tồn tại và Hiệu ứng Fade-out

Để tránh việc người chơi tích trữ quá nhiều mặt trời trên sân hoặc làm đầy bộ nhớ, hệ thống tự động giải phóng tài nguyên sau một khoảng thời gian nhất định:

$$T_{decay} = \begin{cases} 10.000\,ms & \text{(FallingSun)} \\ 12.000\,ms & \text{(Sun từ cây)} \end{cases} \tag{39}$$

Khi đạt giới hạn T_{decay}, hàm `fadeOut()` sẽ giảm dần độ trong suốt của thực thể trước khi xóa hoàn toàn khỏi môi trường game.

38.4 Sự khác biệt trong Vật lý học

Mặc dù cùng kế thừa từ `FallingObject`, tham số khởi tạo `super()` tạo ra sự khác biệt về hành vi:

- **Mặt trời tự nhiên:** `super(2.5, 0, 0, 0, 3000)` → Rơi thẳng đứng, không gia tốc, không xoay.
- **Mặt trời sản xuất:** `super(-3.5, 0.15, Random, 1, 800)` → Bị đẩy ngược lên trên (`vSpeed` âm), chịu tác động của trọng lực và có vận tốc ngang ngẫu nhiên, tạo ra hiệu ứng văng khỏi tâm của cây.

38.5 Kết Luận Sơ Bộ

Hệ thống tài nguyên mặt trời là sự kết hợp nhuần nhuyễn giữa vật lý đơn giản và logic UI. Việc tách biệt các lớp giúp mã nguồn tuân thủ nguyên tắc **Single Responsibility** (Đơn trách nhiệm), giúp dễ dàng cân bằng game (ví dụ: thay đổi giá trị `sunValue` cho các loại mặt trời khác nhau).

39 fallingZombie

39.1 Tóm Tắt

Lớp `fallingZombie` là một lớp bổ trợ kế thừa từ `SpriteAnimator`, chuyên trách việc xử lý hoạt ảnh khi một Zombie bị tiêu diệt. Thay vì biến mất ngay lập tức, đối tượng này tạo ra một hiệu ứng chuyển cảnh bao gồm việc ngã xuống và mờ dần (*fade-out*), giúp tăng tính thẩm mỹ và độ mượt mà cho trải nghiệm người dùng.

39.2 Cơ chế Hoạt ảnh và Hiệu ứng Mờ dần

Lớp này không chứa logic chiến đấu mà tập trung hoàn toàn vào việc quản lý trạng thái hình ảnh cuối cùng của thực thể.

- **Giai đoạn Hoạt ảnh (Frame $\leq$ 7):** Thực hiện chuỗi hình ảnh Zombie đang ngã (`fall`) với tốc độ diễn hoạt là 200ms mỗi khung hình.
- **Giai đoạn Mờ dần (Fade-out):** Sau khi hoàn tất chuỗi hoạt ảnh, đối tượng bắt đầu giảm dần độ trong suốt (*Transparency*).

$$T_{next} = \max(0, T_{current} - 3) \tag{40}$$

Giải thích: Trong mỗi chu kỳ `act`, độ trong suốt (T) giảm đi 3 đơn vị. Nếu giá trị hiệu số nhỏ hơn hoặc bằng 0, độ trong suốt được thiết lập về 0 để tránh lỗi giá trị âm trong Greenfoot.

39.3 Quản lý Vòng đời Đối tượng

Cơ chế tự hủy của `fallingZombie` được kiểm soát chặt chẽ để tối ưu tài nguyên hệ thống:

1. Hệ thống kiểm tra độ trong suốt của hình ảnh trong mỗi frame.
2. Ngay khi `transparency` đạt mức 0 (hoàn toàn biến mất), phương thức `getWorld().removeObject(` sẽ được gọi.
3. Lệnh `return` được sử dụng ngay sau khi xóa để ngăn chặn việc thực thi các dòng mã phía sau lên một đối tượng không còn tồn tại trong thế giới.

39.4 Thông số Kỹ thuật

Bảng 14: Thông số hiệu ứng của fallingZombie

Thuộc tính	**Giá trị**	**Đặc điểm**
`Animation Frames`	8 (0 - 7)	Diễn tả trạng thái ngã xuống
`Frame Duration`	200ms	Tốc độ diễn hoạt trung bình
`Fade Speed`	-3 / act	Giảm độ trong suốt dần dần
`Final State`	Removed	Tự động giải phóng bộ nhớ

39.5 Kết Luận Sơ Bộ

Việc triển khai `fallingZombie` tách biệt với lớp `Zombie` chính giúp mã nguồn trở nên sạch sẽ hơn. Đây là một kỹ thuật hiệu quả trong lập trình Game để xử lý các hiệu ứng hình ảnh phức tạp mà không làm ảnh hưởng đến logic xử lý va chạm hay AI của các thực thể đang hoạt động khác.

40 Tổng kết và Đánh giá

Dự án áp dụng mô hình MVC trên Greenfoot để phân tách minh bạch giữa logic hệ thống và thực thể. Cấu trúc module hóa này giúp tối ưu hóa việc bảo trì và dễ dàng mở rộng các tính năng như Hex-Grid hay cơ chế Merge.